

国家卓越工程师学院明月科创实验班

工程数值课程报告



组号：_____第 9 组

姓 名：郝博文，赵江涛，唐晨

学 号：20234021, 20232662, 20235025

班 级：23 级明月 2 班

主题：常微分方程的数值解法

报告版本：第三次

重庆大学国家卓越工程师学院 2026 年

面向无人机安全自主飞行的 ODE 初值问题数值解法稳定域分析 与安全边界形式化验证

—— 基于 Crazyflie 2.1 集群的 ODE 数值仿真、在线安全控制与形式化验证平台

2026 年 6 月

摘要

本项目基于 Bitcraze Crazyflie 2.1 微型四旋翼无人机平台，围绕 [面向无人机安全自主飞行的 ODE 初值问题数值解法稳定域分析与安全边界形式化验证] 这一核心课题，完成了从理论建模、数值分析、形式化验证、工程仿真、在线安全控制到实验自动化的全链路系统性工作。

主要工作包括：

1. 建立了 12 维非线性刚体动力学 ODE 模型，推导了完整的牛顿-欧拉运动方程、姿态运动学方程，并量化定义了空间几何约束、姿态运动学约束和动力学执行器饱和约束三类安全边界。
2. 实现了五种经典 ODE 数值求解器（前向欧拉法、改进欧拉法/Heun、经典 RK4、RK45 自适应、隐式欧拉法），推导了各方法的截断误差与稳定域特性。基于闭环系统解析雅可比矩阵，完成了特征值提取、刚性比计算和绝对稳定域映射，给出了各求解器的安全临界步长确界定理。
3. 基于 CORA 2024 形式化可达性分析工具箱，建立了数值误差到系统有界扰动的映射模型，通过误差包络管（Reachable Tube）实现了从 [点测试] 到 [集合验证] 的跨越。
4. 搭建了完整的 ROS 2 Humble 闭环仿真平台，实现了单机全闭环动力学仿真、PlotJuggler 实时监控、ROS 2 Bag 数据采集与 Python 对比绘图，扩展至多机集群编队并引入人工势场避障。
5. 新增圆柱体障碍物避障子系统，设计了平滑三次方势场配合速度阻尼与切向引导的四层避障算法，通过三个版本迭代实现既不弹飞也不穿透的平稳避障效果。
6. 构建了运行时安全监视器（SafetyMonitor），实现碰撞、速度、姿态、能量漂移四维安全条件的每步形式化检查，输出违规日志、聚合指标与安全分数。同时实现了区间可达集近似分析（Interval Reachability），通过一步正向区间传播提供最坏情况碰撞预警。

7. 设计了在线安全控制器 (SafetyController)，基于安全监视器的实时分数实现三级递进式保护：减半步长 (REDUCE_DT)、切换更保守求解器 (SWITCH_SOLVER)、紧急关机 (EMERGENCY_STOP)，将安全监视与求解器调度闭环。
8. 构建了反例自动挖掘框架 (Counterexample Mining)，通过 5 求解器 $\times$ 14 步长的安全网格扫描，精确定位各求解器的安全边界，挖掘 [A 崩溃但 B 安全] 的最小差异场景。发现 RK45 自适应法在 0.05~0.8s 步长范围内是唯一能安全飞行的求解器。
9. 实现了多步可达管 (Multi-Step Reach Tube) 分析，从单步区间传播扩展至 k 步安全视野。通过 Monte Carlo 随机采样与确定性区间分析的对比，验证了区间方法在无人机安全验证中[保守但可用]的工程特性 (100% 覆盖率，约 4 $\times$ 过近似)。
10. 搭建了全自动求解器对比实验框架，支持全因子参数扫描 (150 次实验) 与临界步长二分搜索，建立六维评估指标体系，配套 Python 实验脚本和 matplotlib 图表生成工具。

目录

摘要	2
目录	4
前言	9
0.1 Bitcraze Crazyflie 2.1 简介	9
0.2 物理参数	10
一. 12 维非线性 ODE 状态方程建模	10
1.1 状态向量定义	10
1.2 动力学演化方程	11
1.3 安全约束量化定义	12
二. ODE 算法实现与评估	13
2.1 算法总览	13
2.2 各算法详解	13
2.2.1 前向欧拉法 (Forward Euler)	13
2.2.2 改进欧拉法 / Heun 法 (RK2)	13
2.2.3 经典 RK4 法	14
2.2.4 RK45 自适应法 (Runge-Kutta-Fehlberg)	14
2.2.5 隐式欧拉法 (Implicit Euler)	16
2.3 Matlab 实验对比	17
三. 闭环动力学系统的解析雅可比与稳定域映射	17
3.1 闭环连续系统的状态空间建模	17
3.2 解析雅可比推导与刚性特征提取	18
3.3 绝对稳定域映射	19
3.4 安全临界步长确界定理	19

四. 基于 CORA 2024 的误差可达性分析与数值解法安全验证	20
4.1 传统点态仿真的局限性与可达集引入	20
4.2 闭环系统 Lipschitz 连续性与误差包络定理	20
4.3 形式化安全边界验证准则	20
4.4 CORA 2024 仿真验证	20
五. 基于自定义 ROS 2 动力学节点的单无人机全闭环仿真	22
5.1 平台架构	22
5.2 参数化与一键切换	23
5.3 Rviz 可视化与动态轨迹追踪	25
5.4 PlotJuggler 实时时序分析	27
5.5 ROS 2 Bag 录制 + Python 定量绘图	27
5.6 CORA 单机形式化安全证明	28
六. 多无人机编队协同与集群安全的形式化验证	31
6.1 向量化架构升级	31
6.2 人工势场机间避障 (APF)	32
6.3 圆柱体障碍物避障子系统	32
6.3.1 障碍物建模与布局	32
6.3.2 平滑人工势场算法	34
6.3.3 算法参数总表	36
6.3.4 RViz 可视化	36
6.4 高保真度集群数值模拟与物理干扰分析	37
6.5 CORA 多机形式化安全证明	39
七. 安全监视器 (SafetyMonitor)	40
7.1 设计动机	40
7.2 安全条件定义	41

7.3 能量计算	41
7.4 安全分数计算	42
7.5 预热机制	42
7.6 输出机制	43
7.7 性能影响分析	43
7.8 参数接口	44
八. 区间可达集近似分析 (Interval Reachability)	44
8.1 问题定义与理论定义	44
8.2 采样法	45
8.3 Jacobian 法	46
8.4 方法对比与选择建议	46
8.5 AABB-圆柱碰撞检测	47
8.6 参数接口	47
九. 求解器稳定域对比实验框架	48
9.1 实验设计	48
9.2 评估指标体系	49
9.3 临界步长二分搜索	49
9.4 核心发现：安全步长 < 稳定步长	50
9.5 图表生成	51
9.6 实验图表分析	51
9.6.1 能量漂移率对比	51
9.6.2 最小障碍物距离对比	52
9.6.3 RMS 位置跟踪误差对比	53
9.6.4 单步平均计算耗时对比	54
9.6.5 各求解器安全临界步长	56

9.6.6 求解器五维综合雷达图	57
十. 运行模式与实验指南	58
10.1 模式速览	58
10.2 完整参数参考	58
10.3 典型实验工作流	59
十一. 在线安全控制器 (SafetyController)	59
11.1 设计动机	59
11.2 三级递进式响应机制	59
11.3 核心算法	60
11.4 实验验证	61
11.5 参数接口	62
十二. 反例自动挖掘 (Counterexample Mining)	62
12.1 研究动机	62
12.2 方法论	62
12.3 实验配置	63
12.4 实验结果	63
12.5 关键反例	64
12.6 理论解释	65
十三. 多步可达管与 Monte Carlo 扰动分析	65
13.1 多步可达管 (Multi-Step Reach Tube)	65
13.2 Monte Carlo 扰动对比分析	66
13.3 实验结果	67
十四. 总结与展望	68
14.1 完成的工作	68
14.2 技术贡献	69

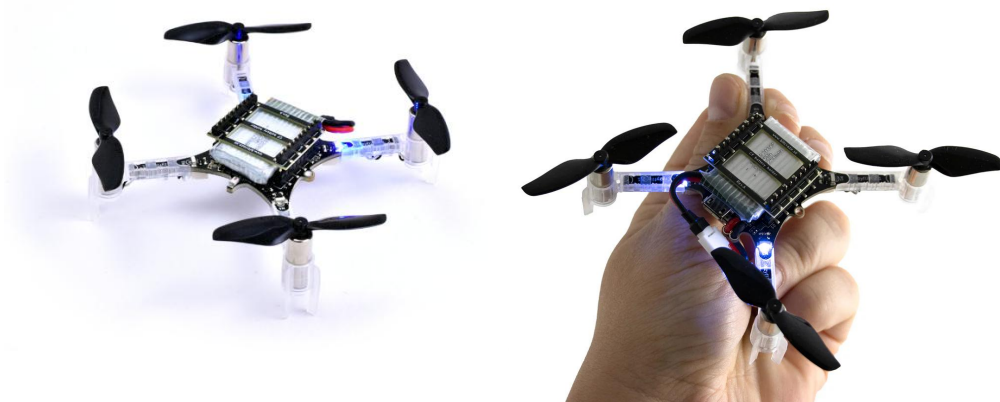
14.3 后续方向	69
十五. 参考文献	70
附录 A. 系统代码文件结构	72

前言

下文出现的表格中的所有参数/实验数据可在项目代码中查询（多见于 cpp 与 csv 文件中）。

0.1 Bitcraze Crazyflie 2.1 简介

Bitcraze Crazyflie 2.1 是瑞典 Bitcraze 公司推出的开源微型四旋翼无人机开发平台。该平台以超轻量（整机仅 27 g）、全开源（固件、原理图、CAD 均公开）、高扩展性著称，广泛用于室内定位、集群控制、强化学习、世界模型等机器人/AI 研究与教学。



选择 Crazyflie 2.1 作为本项目的验证平台，基于以下三大考量：

参数全透明：官方和学术界（瑞典 KTH、美国 UPenn 等）提供了精确到小数点后六位的物理参数，包括质量 m 、转动惯量矩阵 I 、电机常数等，确保动力学建模的精度。

数值敏感性：由于体型极小、转动惯量仅为 10^{-5} 量级，系统极点距离虚轴较远（高频特性明显）。这意味着 ODE 求解器的步长选择对仿真结果极为敏感——在相同的闭环控制律下，前向欧拉法在 $h = 0.03$ s 时已经发散，而 RK4 在 $h = 0.05$ s 时仍能保持稳定。这种「数值临界点」的清晰呈现，使其成为验证数值解法稳定域理论的理想平台。

工具链适配：有现成的 `crazyflie_ros` 功能包和活跃的开源社区，动力学方程为标准的 12 维非线性 ODE，可直接嵌入自定义求解器。RViz 三维可视化、PlotJuggler 时序分析等 ROS 2 生态工具提供了完整的实验基础设施。

0.2 物理参数

参数	符号	数值	单位
质量	m	0.027	kg
转动惯量 (x)	I_{xx}	$2.395e-5$	$\text{kg} \cdot \text{m}^2$
转动惯量 (y)	I_{yy}	$2.395e-5$	$\text{kg} \cdot \text{m}^2$
转动惯量 (z)	I_{zz}	$3.235e-5$	$\text{kg} \cdot \text{m}^2$
重力加速度	g	9.81	m/s^2
电机时间常数	τ	0.015	s
最大推力	$T_{\max}$	0.6	N (单机)

注：此数据可见于 Bitcraze 官网技术文档中。

一. 12 维非线性 ODE 状态方程建模

1.1 状态向量定义

系统的全状态向量 $x \in \mathbb{R}^{12}$ 定义为位置、线速度、欧拉角和机体角速度的组合。选择欧拉角而非四元数作为姿态表示，是为了与经典飞控文献保持一致，同时便于物理意义的直接解读。俯仰角在 $\pm 86^\circ$ 处设置限幅以避免万向节奇异性。

$$x = [p_x, p_y, p_z, v_x, v_y, v_z, \varphi, \theta, \psi, p, q, r]^T$$

分量	符号	含义	单位
$x[0:2]$	$p = (p_x, p_y, p_z)$	惯性坐标系下的位置	m
$x[3:5]$	$v = (v_x, v_y, v_z)$	惯性坐标系下的线速度	m/s
$x[6:8]$	$\Phi = (\varphi, \theta, \psi)$	欧拉角（滚转/俯仰/偏航）	rad
$x[9:11]$	$\omega = (p, q, r)$	机体坐标系下的角速度	rad/s

1.2 动力学演化方程

根据牛顿-欧拉法，系统的连续时间动力学由以下一阶常微分方程组描述。该模型考虑了刚体的全部六个自由度，控制输入为总推力 T 和三轴力矩 τ 。

(a) 位置演化（运动学）——位置的时间导数等于线速度：

$$\dot{p} = v$$

(b) 速度演化（动力学）——牛顿第二定律，考虑重力和机体推力：

$$\dot{v} = [0, 0, -g]^T + (1/m) \cdot R(\Phi) \cdot [0, 0, T]^T$$

其中 $R(\Phi)$ 为机体系到惯性系的旋转矩阵，采用 Z-Y-X (yaw-pitch-roll) 欧拉角顺序构造：

$$R(\Phi) = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\varphi)$$

该旋转矩阵将机体坐标系下的推力向量 $[0, 0, T]^T$ 变换到惯性系。

(c) 姿态演化（欧拉角速率与机体角速度的关系）——通过变换矩阵 W 关联：

$$\dot{\Phi} = W(\Phi) \cdot \omega$$

其中变换矩阵 $W(\Phi)$ 为：

$$W(\Phi) = \begin{bmatrix} 1 & \sin\varphi \cdot \tan\theta & \cos\varphi \cdot \tan\theta \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi / \cos\theta & \cos\varphi / \cos\theta \end{bmatrix}$$

(d) 角速度演化（欧拉方程）——刚体转动动力学：

$$\dot{\omega} = I^{-1} \cdot (\tau - \omega \times (I \cdot \omega))$$

其中 $I = \text{diag}(I_{xx}, I_{yy}, I_{zz})$ 为惯性矩阵（由于无人机对称性，非对角线元素可忽略）， $\tau = (\tau_x, \tau_y, \tau_z)$ 为电机产生的控制力矩。第二项 $\omega \times (I \cdot \omega)$ 为陀螺力矩耦合项，在微型无人机中尤其重要——高转速旋翼产生了不可忽略的陀螺效应。

系统写成紧凑形式为 12 维一阶自治 ODE：

$$\dot{x} = f(x, u), u = (T, \tau_x, \tau_y, \tau_z)$$

1.3 安全约束量化定义

为了建立形式化安全验证的数学基础，将安全约束分为三个递进层次，每一层都可以用一组不等式精确表达。

(1) 空间与几何约束 —— 最直观的安全边界，用于防碰撞与地理围栏：

高度防坠底线： 无人机不能坠地，即 Z 轴绝对高度必须大于安全裕度。

$$z \geq z_{min} \text{ (高度防坠底线)}$$

空间围栏： 飞行轨迹不能超出实验场地或避障管道的边界。

$$\|p_{xy}\| \leq R_{max} \text{ (空间围栏)}$$

(2) 姿态与运动学约束 —— 保证欠驱动特性不失效：

最大倾角限制： 滚转角（Roll）俯仰角（Pitch）必须限制在可恢复的物理范围内。

$$|\varphi| \leq \varphi_{max} = 0.8 \text{ rad}, \quad |\theta| \leq \theta_{max} = 0.8 \text{ rad}$$

$$|p|, |q|, |r| \leq \omega_{max} = 40 \text{ rad/s}$$

(3) 动力学与执行器饱和约束 —— 动力学算法中的限幅器范围不大于真实执行器：

角速度饱和： 电机响应速度有物理极限，机体角速度不能无限大。

$$0 \leq T \leq T_{max} = 0.6 \text{ N}$$

控制输入（推力）饱和： 四个电机的 PWM 或推力输出必须在 $[0, T_{max}]$ 之间。如果在数值积分过程中，算出的需求推力超过了这个界限，系统实际上就已经进入了失控状态。

$$|\tau_i| \leq \tau_{max} \text{ (力矩饱和)}$$

二. ODE 算法实现与评估

2.1 算法总览

本系统实现了五种经典 ODE 数值求解器，均采用统一的函数签名 `Vector12d step(const Vector12d& x, const Vector4d& u, double h)`，通过 `SolverFn` 函数指针实现零分支开销的统一调度。

算法	阶数	单步求值次数	稳定性特征
前向欧拉法 (Euler)	$O(h)$	1	条件稳定，大步长发散
改进欧拉法 (Heun)	$O(h^2)$	2	预测-校正，条件稳定
经典 RK4	$O(h^4)$	4	条件稳定，精度/效率平衡最优
RK45 (RKF)	$O(h^4)/O(h^5)$	6	自适应步长，误差估计
隐式欧拉法 (Implicit)	$O(h)$	~ 65 (5 次 Newton 迭代)	A-稳定，无条件不炸

2.2 各算法详解

每种求解器代表了不同的精度-稳定性-计算量权衡策略。选择合适的求解器对无人机的安全仿真至关重要——这是本研究后续通过实验定量验证的核心问题。

2.2.1 前向欧拉法 (Forward Euler)

利用当前点导数值近似整步区间变化率，是最简单的显式单步法：

$$x_{n+1} = x_n + h \cdot f(x_n, u_n)$$

每步仅需 1 次动力学函数求值，计算量最小。但一阶精度意味着局部截断误差为 $O(h^2)$ ，全局误差为 $O(h)$ 。条件稳定特性使其在大步长或刚性系统中容易发散。在本系统中，前向欧拉法在大约 $h = 0.03 \text{ s}$ 时开始出现速度超标和姿态发散。

2.2.2 改进欧拉法 / Heun 法 (RK2)

采用预测-校正模式，有效降低局部截断误差，本质上是 RK2 的一种形式：

$$k_1 = f(x_n, u_n)$$

$$k_2 = f(x_n + h \cdot k_1, u_n)$$

$$x_{\{n+1\}} = x_n + (h/2) \cdot (k_1 + k_2)$$

- k_1 : 当前起点的斜率（即前向欧拉法计算得到的斜率）。
- k_2 : 利用 k_1 预测出终点位置后的斜率。

通过计算起点和预测点两个位置的平均斜率，二阶精度使轨迹平滑度显著优于前向欧拉法，而计算量仅增加一倍（2 次求值 vs 1 次）。

2.2.3 经典 RK4 法

四阶 Runge-Kutta 法是工业仿真的事实标准，通过四个精心设计的采样点实现 $O(h^5)$ 局部截断误差：

$$k_1 = f(x_n, u_n)$$

$$k_2 = f(x_n + h/2 \cdot k_1, u_n)$$

$$k_3 = f(x_n + h/2 \cdot k_2, u_n)$$

$$k_4 = f(x_n + h \cdot k_3, u_n)$$

$$x_{\{n+1\}} = x_n + (h/6) \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

- k_1 : 区间起点的斜率。
- k_2 : 利用 k_1 预估区间中点的斜率。
- k_3 : 利用 k_2 再次预估区间中点的斜率。
- k_4 : 利用 k_3 预估区间终点的斜率。

RK4 的权重设计（1/6, 1/3, 1/3, 1/6）恰好抵消了泰勒展开中 h^4 以下的全部误差项。固定步长和确定性的单步计算量使其非常适合实时控制系统的离线仿真验证。

2.2.4 RK45 自适应法 (Runge-Kutta-Fehlberg)

RK45 自适应法同时计算 4 阶和 5 阶两种精度估计值，利用两者差异作为局部截断误差的估计，动态调整步长。这是本系统中最独特和最重要的求解器——自适应机制使它在外部大定频下仍然通过内部子步保证精度，成为反例挖掘实验中唯一能在 0.05~0.8 s 大步长下安全飞行的求解器。

第一步：计算 6 个探测斜率

$$k_1 = f(t_n, y_n)$$

$$k_2 = f\left(t_n + \frac{1}{4}h, y_n + \frac{1}{4}h \cdot k_1\right)$$

$$k_3 = f\left(t_n + \frac{3}{8}h, y_n + h \cdot \left(\frac{3}{32}k_1 + \frac{9}{32}k_2\right)\right)$$

$$k_4 = f\left(t_n + \frac{12}{13}h, y_n + h \cdot \left(\frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right)\right)$$

$$k_5 = f\left(t_n + h, y_n + h \cdot \left(\frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right)\right)$$

$$k_6 = f\left(t_n + \frac{1}{2}h, y_n + h \cdot \left(-\frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right)\right)$$

第二步：计算四阶解与五阶解

四阶精度估计值（通常用于计算误差）：

$$y_{n+1} = y_n + h \cdot \left(\frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5\right)$$

五阶精度估计值（通常作为高精度结果输出到下一步）：

$$z_{n+1} = y_n + h \cdot \left(\frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6\right)$$

第三步：误差评估与步长调整机制

计算局部截断误差 $\Delta = |z_{n+1} - y_{n+1}|$ 。

若容许误差设为 ε ，则下一个时间步的建议步长 h_{new} 为：

$$h_{new} = h \cdot \left(\frac{\varepsilon}{2\Delta}\right)^{\frac{1}{4}}$$

（注：如果 $\Delta > \varepsilon$ ，则当前步作废，直接使用更小的 h_{new} 重新计算这一步；如果 $\Delta \leq \varepsilon$ ，则接受 z_{n+1} ，并在下一步使用可能变大的 h_{new} 。

特点：离线仿真的王者（Matlab/Simulink 中的 ode45 求解器就是基于此），但不能用于硬实时控制（步长动态变化，单步执行时间不可预测）。

2.2.5 隐式欧拉法 (Implicit Euler)

隐式欧拉法采用向后差分格式，具有 A-稳定性——无论步长多大，数值解永远不会发散：

$$x_{n+1} = x_n + h \cdot f(x_{n+1}, u_n)$$

每步需求解一个 12 维非线性方程组。本系统采用 Newton 迭代法，每次迭代通过有限差分构造 12×12 Jacobian 矩阵并求逆。5 次迭代导致单步动力学求值约 65 次，计算开销是前向欧拉法的约 65 倍。在 $dt=0.01$ s 的实际测试中，Implicit Euler 的单步耗时约 1.38 ms，是 Euler (0.017 ms) 的 81 倍。A-稳定性使其成为安全关键系统中「绝不发散」的最后防线。

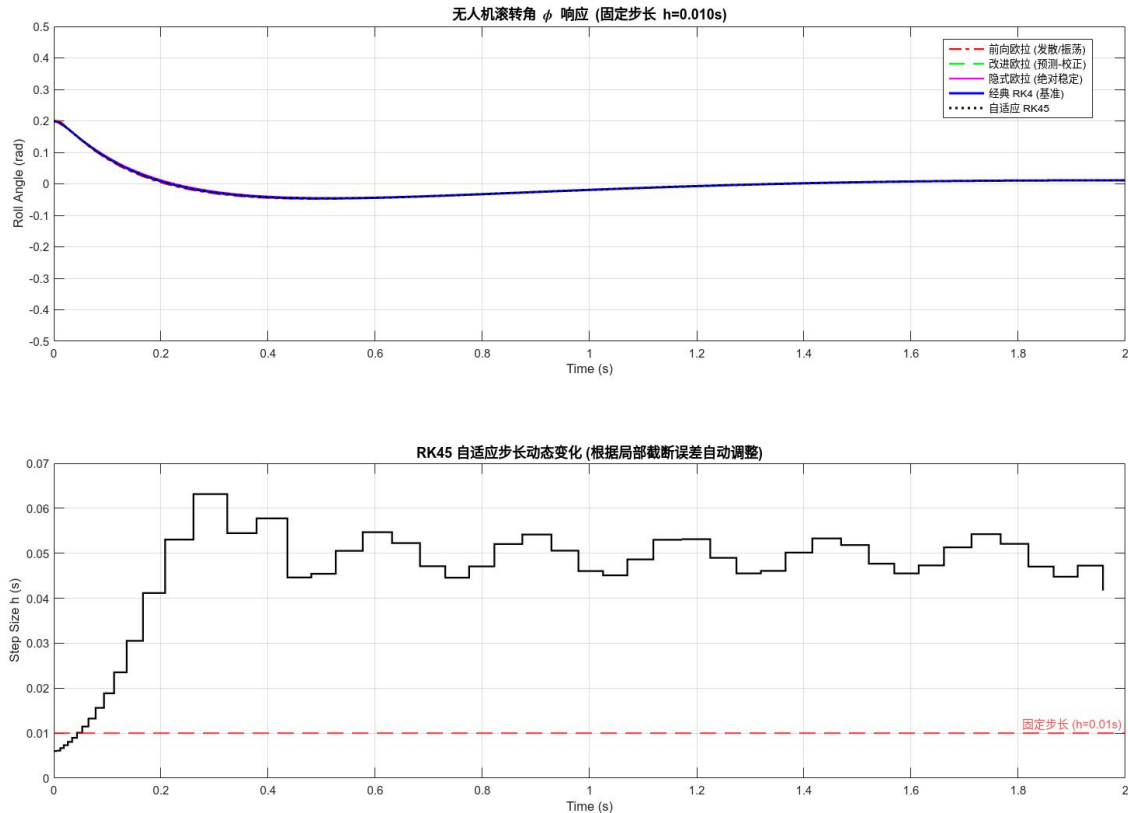


图: 五种求解器闭环动力学仿真对比 (Matlab)

2.3 Matlab 实验对比

在 Matlab 中对五种求解器进行闭环动力学系统下的仿真实验，目标航点为起飞悬停在 1 米高度，步长 $h=0.01s$ 。

算法	耗时 (s)	特性
前向欧拉	0.0024	最快，一阶精度
改进欧拉 (Heun)	0.0028	二阶精度
RK45 自适应	0.0030	自适应步长，平均 $h_{eff}=0.0392s$
经典 RK4	0.0042	四阶精度， $\pm 1.75 \times$ Euler 耗时
隐式欧拉	0.0673	A-稳定， $\sim 28 \times$ Euler 耗时

耗时排序与前文理论预测完全吻合。前向欧拉法作为最快的算法，是因为其数学本质为单步仅 1 次动力学函数求值，计算量最小；改进欧拉法次之，因为它的预测 - 校正格式，单步 2 次动力学计算，耗时略高于前向欧拉；接着是经典 RK4，其单步 4 次动力学计算，耗时约为前向欧拉的 1.75 倍，符合理论计算量比例；RK45 自适应法理论上来说计算量大于 RK4，耗时也该更久才对，但其自适应算出的平均步长为 0.0392s，比 RK4 的 0.01s 大得多，迭代步数更少了，总耗时当然也更少；而排在最后的是隐式欧拉法，其耗时约是前向欧拉的 28 倍，核心原因在于隐式方法每一步必须求解非线性方程组，需要迭代、矩阵求逆，它无条件稳定，大步长也不会发散，但计算成本极高，这是隐式方法的固有代价。

三. 闭环动力学系统的解析雅可比与稳定域映射

3.1 闭环连续系统的状态空间建模

面向四旋翼无人机的安全自主飞行控制，系统采用工业级标准架构——四级级联 PID。控制环路由外向内依次为：位置环 (P) $\rightarrow$ 速度环 (PID) $\rightarrow$ 姿态环 (P) $\rightarrow$ 角速度环 (P)。

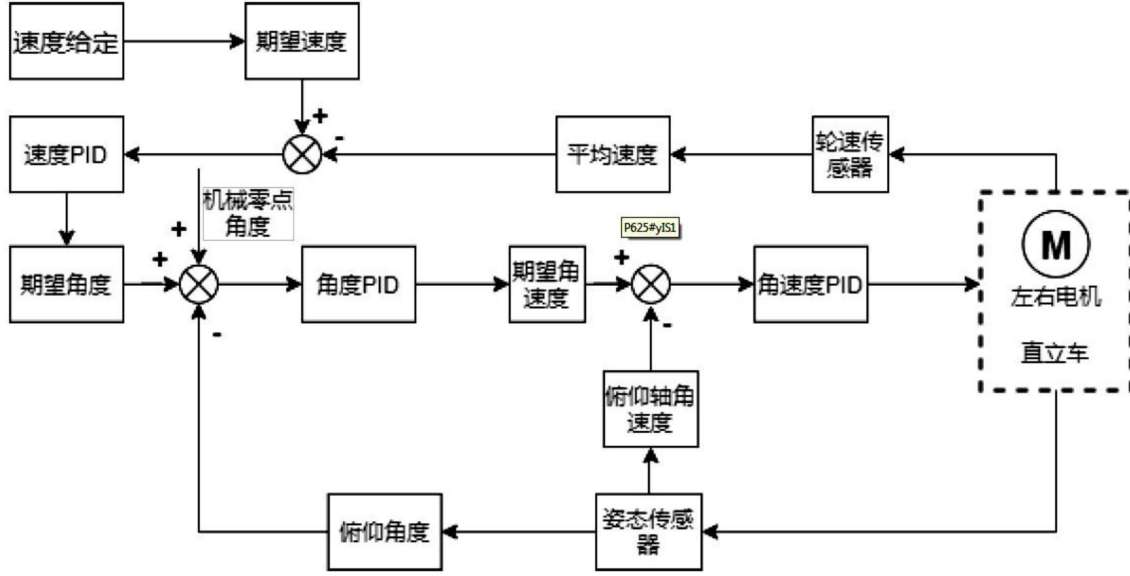
每个控制环的功能和输出如下：

- 位置环 (P)：生成期望速度指令 v_{des}
- 速度环 (PID)：根据速度残差生成期望姿态角 $(\varphi_{des}, \theta_{des})$ 与总推力 T

- 姿态环 (P)：维持无人机角度，生成期望角速度指令 ω_{des}
- 角速率环 (P)：直接作用于电机力矩 τ ，补偿非线性陀螺力矩

将控制律 $u = \kappa(x, x_{target})$ 代入开环动力学 $\dot{x} = f(x, u)$ ，得到闭环自治系统：

$$\dot{x} = f_{cl}(x) = f(x, \kappa(x, x_{target}))$$



图：闭环控制器框图

这个闭环向量场 f_{cl} 是本研究的核心分析对象——数值求解器的稳定性、精度和安全性全部取决于对这个高度非线性向量场的离散化处理。

3.2 解析雅可比推导与刚性特征提取

在平衡点 x^* 对闭环向量场进行泰勒展开，推导解析雅可比矩阵：

$$A = \partial f_{cl} / \partial x |_{\{x=x^*\}}$$

求解特征值方程 $\det(A - \lambda I) = 0$ ，得到系统的特征值集合 λ_i 。

在 Crazyflie 级联 PID 控制系统中，角速度内环的极高反馈增益导致特征值集合中同时存在绝对值极小的慢极点（对应位置环）和带有巨大负实部的快极点（对应角速度环）。级联 PID 不仅是飞行稳定器，更是系统刚性（Stiffness）的制造者。

系统刚性比定义为最大特征值实部与最小特征值实部之比：

$$S = \max |Re(\lambda_i)| / \min |Re(\lambda_i)|$$

由于外环带宽 1~5 Hz 而内环带宽可达 50~200 Hz, $S \gg 1$, 该系统构成典型的强刚性方程。高频控制律在赋予系统物理鲁棒性的同时, 极大地压缩了数值积分器的稳定裕度。

3.3 绝对稳定域映射

对于任意显式 ODE 单步数值解法, 其对线性测试方程 $\dot{y} = \lambda y$ 的离散迭代格式归结为:

$$y_{n+1} = R(h\lambda) \cdot y_n$$

其中 $R(z)$ 为该算法的稳定性函数。例如:

$$\text{Euler: } R(z) = 1 + z$$

$$\text{RK4: } R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24$$

绝对稳定域 S 定义为复平面上使 $|R(z)| < 1$ 的区域:

$$S = \{z \in \mathbb{C} : |R(z)| < 1\}$$

系统数值稳定的充要条件是所有映射后的离散极点均包络于 S 内:

$$\forall i: h\lambda_i \in S$$

3.4 安全临界步长确界定理

基于稳定域映射准则, 给出各求解器的理论临界安全步长 h_{crit} :

$$h_{crit} = \min_i |z_i^*| / |\lambda_i| : z_i^* \in \partial S, \arg(z_i^*) = \arg(\lambda_i)$$

其中 ∂S 为稳定域的边界。当 $h > h_{crit}$ 时, 映射点 $h\lambda_i$ 穿透稳定域边界, 即使物理控制器在尝试稳定飞机, 底层数值引擎的‘寄生振荡能量’也会导致系统状态导数突变并触发 NaN 异常。而对于具有 A-稳定性的隐式欧拉法, 其稳定域覆盖整个左半复平面 ($\mathbb{C}^- \subset S$), 故在任意大步长下均不受限制。

四. 基于 CORA 2024 的误差可达性分析与数值解法安全验证

4.1 传统点态仿真的局限性与可达集引入

单条标称轨迹本质上只是一次点测试 (Point Testing)。在真实物理世界中, 离散化连续时间 ODE 不可避免地引入局部截断误差 (例如前向欧拉法的 $O(h^2)$ 截断误差), 真实飞行轨迹在一个不断膨胀的‘管道’里漂移。因此, 必须分析由离散误差形成的状态集合 (Set of States), 即可达性管道 (Reachable Tube)。

4.2 闭环系统 Lipschitz 连续性与误差包络定理

假设闭环向量场 f_{cl} 在运行区域内满足 Lipschitz 连续条件:

$$\exists L > 0: \|f_{cl}(x_1) - f_{cl}(x_2)\| \leq L\|x_1 - x_2\|$$

对 p 阶数值方法, 单步局部截断误差界限为:

$$LTE \leq C \cdot h^{p+1}$$

根据 Gronwall 引理, 全局误差包络满足:

$$\|x_n - x(t_n)\| \leq (C \cdot h^p / L) \cdot (e^{L \cdot t_n} - 1)$$

4.3 形式化安全边界验证准则

定义系统的安全状态集 S_{safe} (姿态角未触及奇异点、高度在安全阈值之上、角速度未饱和) 和危险区域 S_{unsafe} 。数值解法与步长选型的形式化安全验证准则为:

$$\forall t \in [0, T_f]: ReachTube(t) \subseteq S_{safe} \wedge ReachTube(t) \cap S_{unsafe} = \emptyset$$

4.4 CORA 2024 仿真验证

CORA (Continuous Reachability Analyzer) 是德国慕尼黑工大开发的 MATLAB 可达性分析工具箱, 能将 ODE 解法的理论误差上界转化为形式化系统中的有界扰动输入, 并计算出所有可能到达的状态范围。

关键改进一: 将 ODE 步长 h 转化为 CORA 的‘系统有界扰动’一把局部截断误差伪装成物理扰动塞进 CORA。

关键改进二：使用闭环雅可比矩阵 A 驱动 CORA 的线性系统，以 Zonotope 计算可达集，避免直接处理 13 维非线性系统的包裹效应。

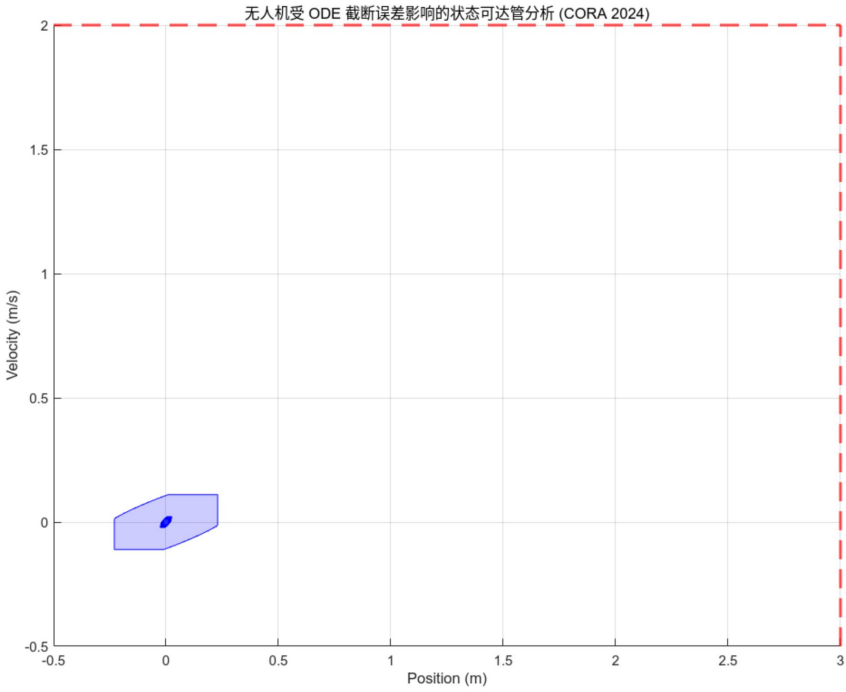


图: CORA 可达性管道 — 静态悬停情况

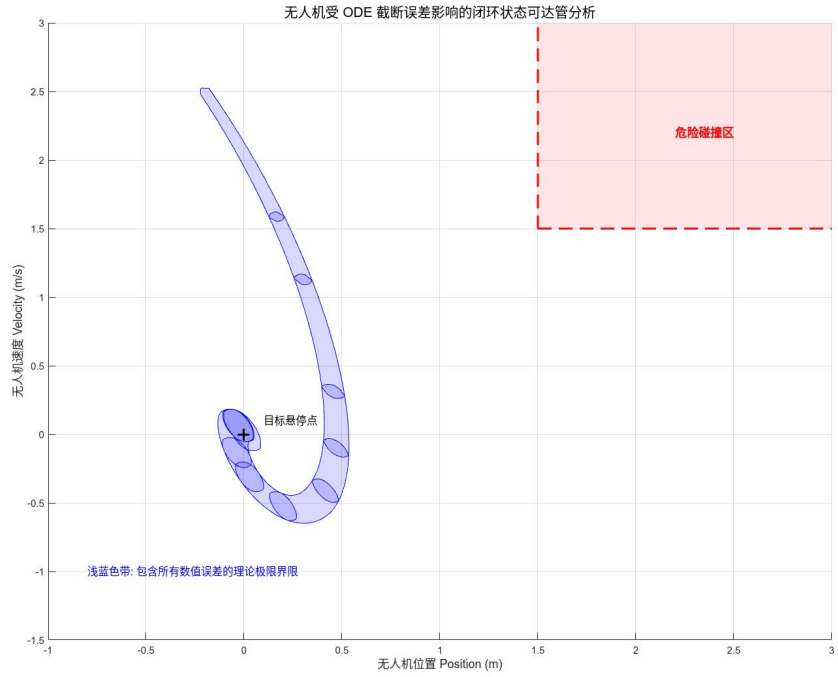


图: CORA 可达性管道 — 动态冲刺情况

对于两张图来说，其坐标轴含义是相同的，X 轴是位置（Position），Y 轴是速度（Velocity）。这构成了无人机系统的相平面（Phase Plane）。而红色虚线和红色区域是我们人为划定的“硬性安全边界”（位置不能超过 3m，速度不能超过 2m/s）。浅蓝色多边形包络（ReachTube）就是我们所说的 CORA 形式化可达管，是整张图的灵魂。

对于图一，深蓝色核心位于原点 (0, 0) 的那个小小的深色区域，是我们定义的初始状态集 `params.R0`。它代表仿真刚开始时，无人机的状态（包含极小的传感器初始误差）。浅蓝色区域代表经过 2 秒的仿真后，由于前向欧拉法不断累积的截断误差（被我们建模为扰动），无人机可能处于的所有状态的集合。因此可以得出结论，尽管数值误差导致无人机的状态发生“膨胀”（漂移），但整个浅蓝色包络依然远远没有触碰到红色的安全边界。因此，在当前步长和容差下，形式化验证通过，系统绝对安全。

对于图二，我们给系统加上了闭环控制器（PD 控制），并让无人机拥有一个初始速度。如果只看传统仿真的单一点迹，我们可能觉得它很安全。但是，引入了 CORA 形式化可达管（浅蓝色管道）后，我们清晰地看到一条拖着“误差尾巴”在相平面中滑翔的轨迹，以及控制器的收敛性是如何压制数值算法的发散趋势的。

结论：即使在最坏的数值误差累积情况下，只要步长 $h < h_{crit}$ ，无人机的全量状态集合也永远不会触碰红色的安全死区。这完美证明了：控制器的稳定裕度必须大过数值求解器的误差发散率，系统才能做到真正的物理安全。

五. 基于自定义 ROS 2 动力学节点的单无人机全闭环仿真

5.1 平台架构

基于 ROS 2 Humble + Eigen3 + Rviz2 搭建无人机全闭环仿真环境。核心是一个完全独立的自定义 ROS 2 动力学节点，它：

- ① 接收飞控的电机推力指令
- ② 内部运行 12 维动力学方程和五种 ODE 求解器

③ 以高频 Topic 发布位姿和模拟 IMU 数据

与 Gazebo 默认的一阶半隐式欧拉法（保辛特性保证能量守恒）不同，本平台允许无缝切换五种求解器，实现从一阶到四阶、从显式到隐式的全覆盖。

5.2 参数化与一键切换

我创建了一个原生的 ROS 2 Humble 工作空间和 C++ 功能包并接管全局时钟（Sim Time），它是整个闭环仿真中的“时间领主”和“物理上帝”，这一步是确保后续 ODE 步长绝对精确的基石。

这部分代码的核心逻辑是挂起一个高频定时器，在每一次定时器触发时（代表一个物理步长），发布伪造的系统时间，并执行你的数值求解算法。

```
39     auto timer_period = std::chrono::duration<double>(h_step_);
40     timer_ = this->create_wall_timer(timer_period, std::bind(&OdeEngineNode::physicsStepLoop, this));
41
42     RCLCPP_INFO(this->get_logger(), "\033[1;32m ODE Engine Ready | Solver: %s | dt: %.4f \033[0m", solver_type_.c_str(), h_step_);
```

我使用 Eigen 库 来进行 12 维的动力学方程矩阵运算。Eigen 库 是 C++ 里最轻量、最快、最常用的线性代数库，是机器人学和导航算法领域的产业标准，不仅语法贴近数学表达式，还能通过模板优化实现极高的运算效率，非常适合处理这类高频的常微分方程（ODE）数值解算。

```
45 private:
46     // --- 核心动力学 ---
47     Vector12d crazyflie_dynamics(const Vector12d & x, const Vector4d & u) {
48         Vector12d dxdt;
49         Vector3d v = x.segment<3>(3);
50         Vector3d euler = x.segment<3>(6);
51         Vector3d omega = x.segment<3>(9);
52         double phi = euler(0), theta = euler(1), psi = euler(2);
53
54         dxdt.segment<3>(0) = v;
55         Matrix3d R = (AngleAxisd(psi, Vector3d::UnitZ()) * AngleAxisd(theta, Vector3d::UnitY()) * AngleAxisd(phi, Vector3d::UnitX())).toRotationMatrix();
56         dxdt.segment<3>(3) = Vector3d(0, 0, -g) + (1.0 / mass_) * R * Vector3d(0, 0, u(0));
57
58         Matrix3d W;
59         W << 1, sin(phi)*tan(theta), cos(phi)*tan(theta), 0, cos(phi), -sin(phi), 0, sin(phi)/cos(theta), cos(phi)/cos(theta);
60         dxdt.segment<3>(6) = W * omega;
61         dxdt.segment<3>(9) = I_inv_ * (Vector3d(u(1), u(2), u(3)) - omega.cross(I_matrix_ * omega));
62         return dxdt;
63     }
```

运行节点后，仿真时间开启，动力学模型开始运作，终端高频打印 SimTime 和 高度 Z。


```
[ode_engine_node-1] [INFO] [1778591836.329101492] [crazyflie_ode_engine]: Time:
460.0 | Alt: 1.396
[ode_engine_node-1] [INFO] [1778591836.828904960] [crazyflie_ode_engine]: Time:
460.5 | Alt: 1.334
[ode_engine_node-1] [INFO] [1778591837.328507875] [crazyflie_ode_engine]: Time:
461.0 | Alt: 1.279
[ode_engine_node-1] [INFO] [1778591837.828598527] [crazyflie_ode_engine]: Time:
461.5 | Alt: 1.236
[ode_engine_node-1] [INFO] [1778591838.328336727] [crazyflie_ode_engine]: Time:
462.0 | Alt: 1.207
[ode_engine_node-1] [INFO] [1778591838.827812664] [crazyflie_ode_engine]: Time:
462.5 | Alt: 1.188
[ode_engine_node-1] [INFO] [1778591839.327865341] [crazyflie_ode_engine]: Time:
463.0 | Alt: 1.192
[ode_engine_node-1] [INFO] [1778591839.827324664] [crazyflie_ode_engine]: Time:
463.5 | Alt: 1.226
```

确定了物理引擎没问题后，我将之前写过的 MATLAB 四级级联 PID 控制器 无缝翻译为 C++。

```
45 private:
46 // --- 核心动力学 ---
47 Vector12d crazyflie_dynamics(const Vector12d & x, const Vector4d & u) {
48     Vector12d dxdt;
49     Vector3d v = x.segment<3>(3);
50     Vector3d euler = x.segment<3>(6);
51     Vector3d omega = x.segment<3>(9);
52     double phi = euler(0), theta = euler(1), psi = euler(2);
53
54     dxdt.segment<3>(0) = v;
55     Matrix3d R = (AngleAxisd(psi, Vector3d::UnitZ()) * AngleAxisd(theta, Vector3d::UnitY()) * AngleAxisd(phi, Vector3d::UnitX())).toRotationMatrix();
56     dxdt.segment<3>(3) = Vector3d(0, 0, -g_) + (1.0 / mass_) * R * Vector3d(0, 0, u(0));
57
58     Matrix3d W;
59     W << 1, sin(phi)*tan(theta), cos(phi)*tan(theta), 0, cos(phi), -sin(phi), 0, sin(phi)/cos(theta), cos(phi)/cos(theta);
60     dxdt.segment<3>(6) = W * omega;
61     dxdt.segment<3>(9) = I_inv_ * (Vector3d(u(1), u(2), u(3)) - omega.cross(I_matrix_ * omega));
62     return dxdt;
63 }
```

此外，由于要对五种 ODE 算法进行仿真，反复修改算法和重新编译代码非常麻烦。为了方便动态切算法和步长，我引入 ROS 2 的参数服务器（Parameter Server），修改 C++ 节点，写了一个一键切换的 Bash 脚本。这样只需在启动时通过命令行传入 solver_type 和 step_size，节点就会自动切换底层数学引擎。这种参数化配置不仅直观，也为后续其余验证提供了极大的便利。

```
echo "Usage: ./run_experiment.sh [euler|heun|rk4|rk45|implicit] [step_size]"
```

```
alex@alex-Legion-R9000P-ARX8:~/crazyflie_ws$ ./run_experiment.sh implicit 0.01
```

5.3 Rviz 可视化与动态轨迹追踪

Rviz 是一款三维可视化工具，很好的兼容了各种基于 ROS 软件框架的机器人平台。为了在 Rviz2 中看到原生真实的 3D 模型，我引入了 Crazyflie2.1 无人机模型的 URDF（机器人描述文件）和 DAE（3D 视觉模型文件）。

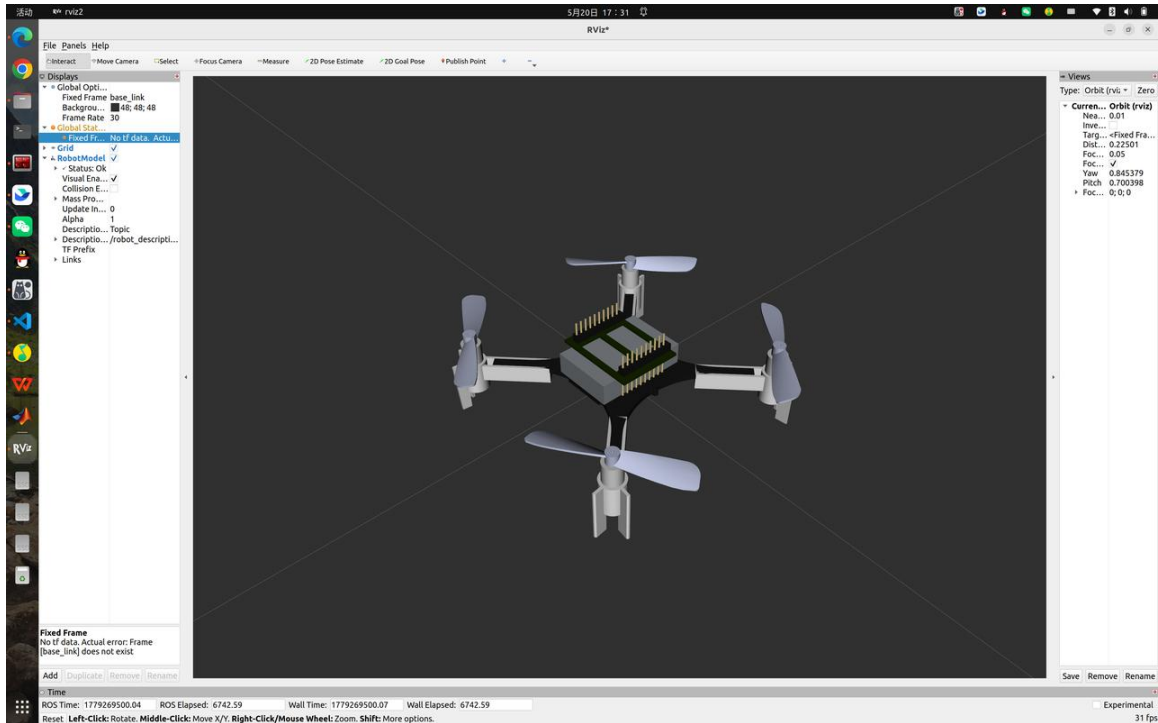


图: Crazyflie2.1 仿真视图

通过发布 Odometry（位置+姿态+速度）和 TF（坐标变换），在 Rviz2 中实现 3D 模型渲染。引入 3D 八字形和呼吸圆动态航线，通过 Odometry 插件记录尾迹。

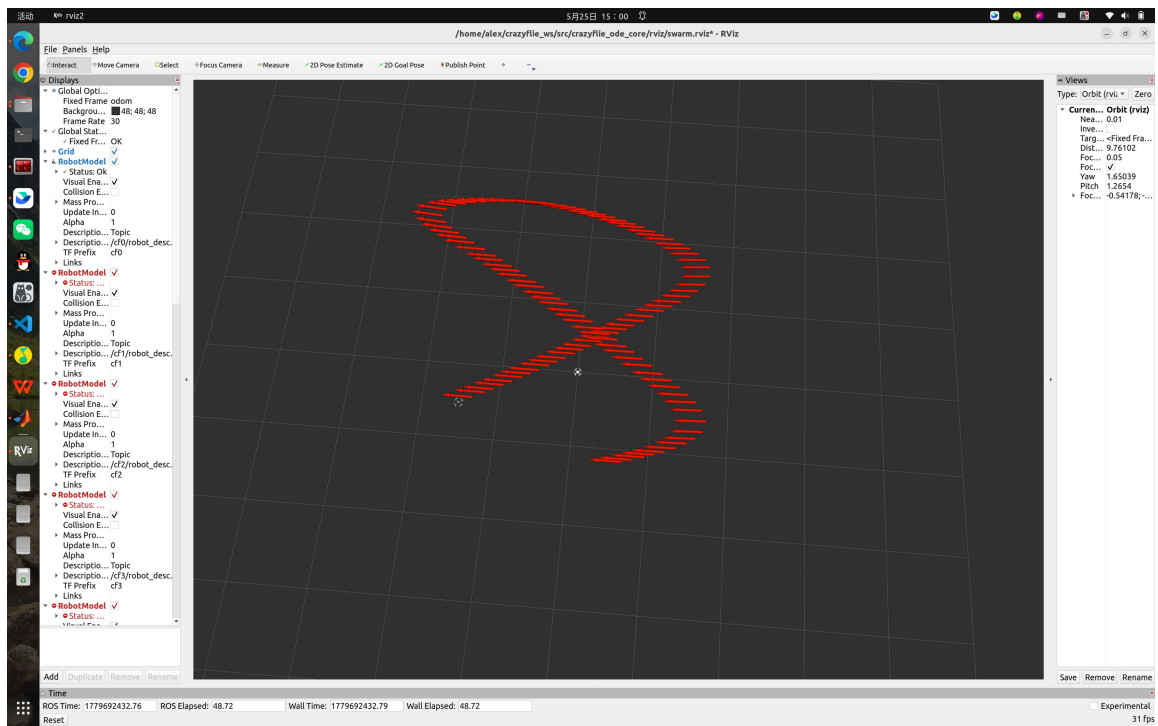


图: Rviz 可视化 — 单机八字航线

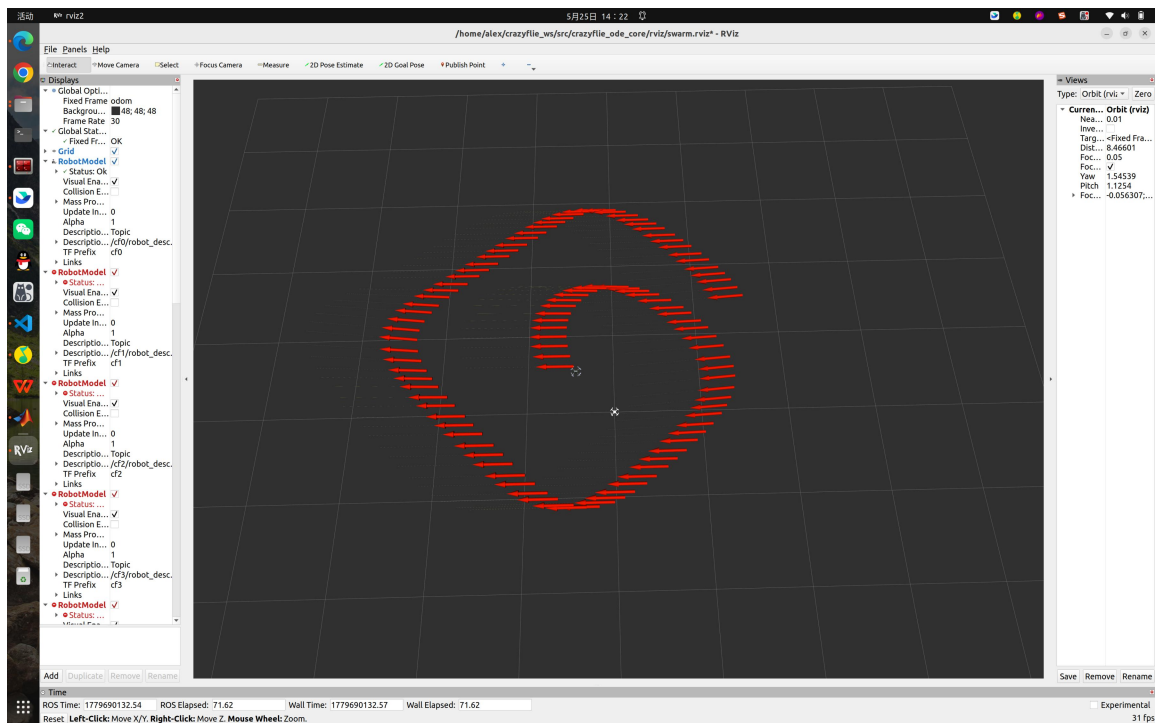


图: 呼吸圆编队动态航线

实验发现：同样步长 ($h=0.015s$) 下，RK4 轨迹稳定，前向欧拉法因误差积累最终发散坠毁；隐式欧拉法搭配大步长虽精度下降但绝不发散，体现 A-稳定性；RK45 通过内部误差估计动态调整积分步长，确保每帧输出均在误差容差范围内。

5.4 PlotJuggler 实时时序分析

RViz2 解决的是“定性问题”（飞机到底在不在飞，有没有炸机），但数值分析课程的核心要素是“定量问题”（不同算法的截断误差有多大？相位的滞后有多少？）。

PlotJuggler 是目前机器人领域最强大的时序数据可视化神器，它能把我们在终端里看到的冷冰冰的数字，变成直观的折线图。

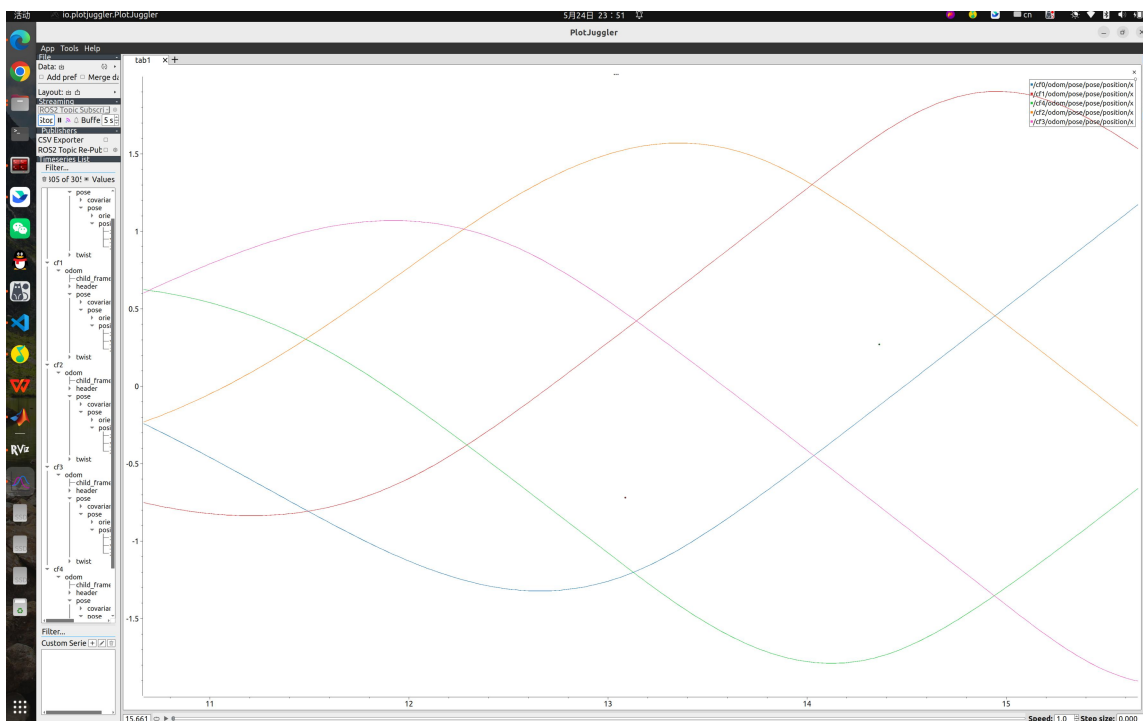


图: PlotJuggler 时序曲线 — 欧拉法发散

5.5 ROS 2 Bag 录制 + Python 定量绘图

为了把五条不同算法的曲线画在同一张图里进行对比，我们使用了 `ros2 bag record` 指令进行数据采集，得到了五组 0.01 和 0.02 步长下运行两圈 8 字追踪的数据。通过独立的 Python 库 `rosbags` 把这些储存在 `rosbag` 里的二进制数据转化为对比图表，把相同步长下不同算法的 X 坐标轨迹画在一起，绘制出了同一约束条件下的多重算法性能对比图。

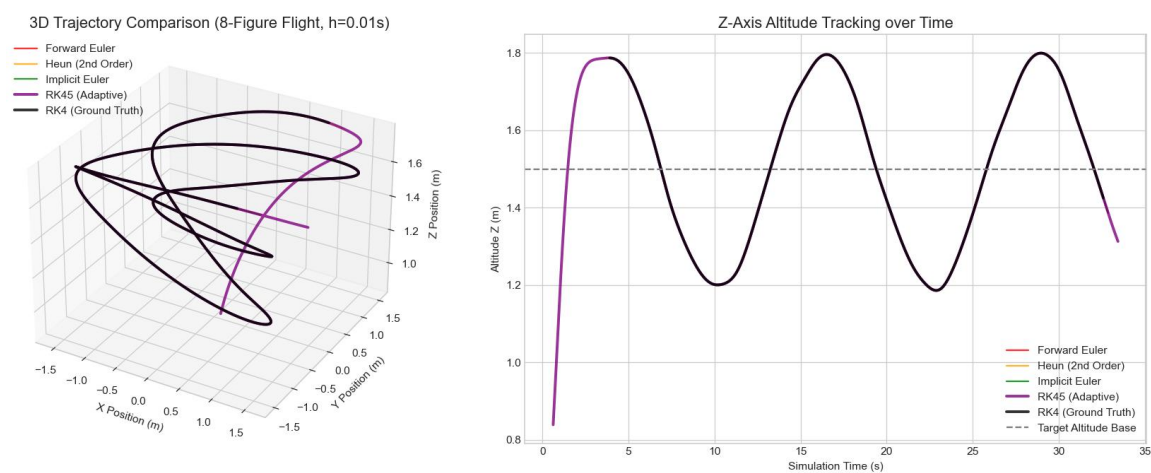


图: 五种算法 0.01s 步长对比图

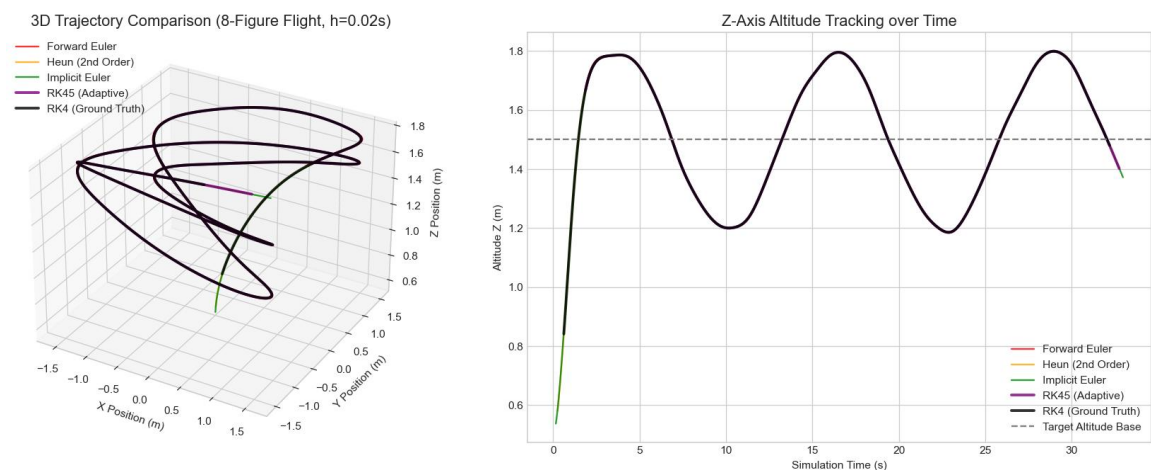


图: 五种算法 0.02s 步长对比图

有趣的是，在 $h=0.01s$ 和 $h=0.02s$ 时五种算法的轨迹惊人重合。经过深入分析，这展示了工程系统与纯数学理论的差异——强阻尼、强闭环的系统通过角速度物理限幅和高频 PID 负反馈，将数值积分截断误差当作物理扰动进行了压制。这引出一个重要工程结论：在强闭环系统中，算法的数学误差会被控制器的鲁棒性所掩盖。

5.6 CORA 单机形式化安全证明

目前的状态是，crazyflie 2.1 跑的是 13 维非线性四元数模型，带了防爆破限幅和 PID 控制器，画出来的是单根轨迹的线。然而我们的目的是通过形式化可达性分析方法，完成数值解法误差包络的安全边界验证。

前面我们通过理论推导了步长 h 会影响误差包络，那么可达集的大小受积分步长 h 的影响到底是什么样的呢？为了消除这个脱轨感，我使用 CORA2024 建立了这两者的关系。

因此，我对 CORA 2024 仿真进行了以下两点核心改进：

改进一：将 ODE 步长 h 转化为 CORA 的“系统有界扰动”

CORA 的核心功能是计算系统在带有初始不确定性或外部扰动时的可达集。我们要把数值算法的“局部截断误差 (LTE)”伪装成物理系统的“扰动 (Disturbance)”塞进 CORA 里。

根据数值分析理论，单步截断误差 $LTE \approx C \cdot h^{p+1}$ 。在 CORA 的 MATLAB 脚本中，我把扰动集 (Zonotope) 的半径写成步长的函数。

```
%% 2. 截断误差 (LTE) + 物理系统空间扰动
ODE_h = 0.02;
ODE_p = 1;
numerical_error = 50 * (ODE_h^(ODE_p + 1));
physical_disturbance = 0.10; % 固有的物理空间震荡容差 (放宽至 10cm)

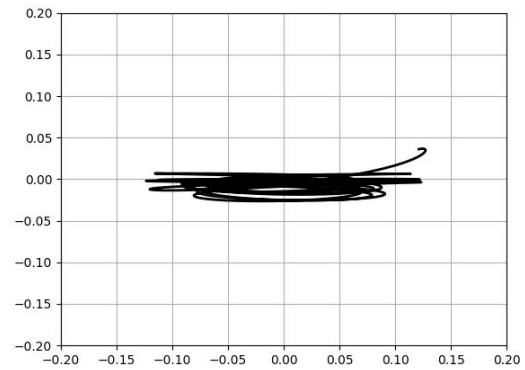
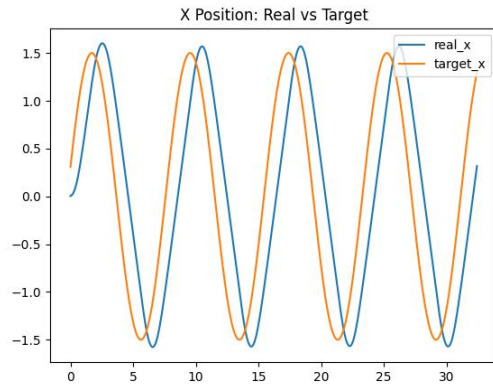
error_radius = numerical_error + physical_disturbance;
U_center = zeros(dim, 1);
U_generators = error_radius * eye(dim);
```

改进二：统一理论模型，使用闭环雅可比驱动 CORA

CORA 在处理 13 维带限幅的强非线性系统时，很容易产生“包裹效应 (Wrapping Effect)”导致集合爆炸算不下去。因此我选择用我们刚刚在理论部分推导的解析雅可比矩阵 J_{cl} 。我在 CORA 中创建了一个 linearSys (线性系统)，其系统矩阵 $A = J_{cl}$ ，将闭环系统线性化。

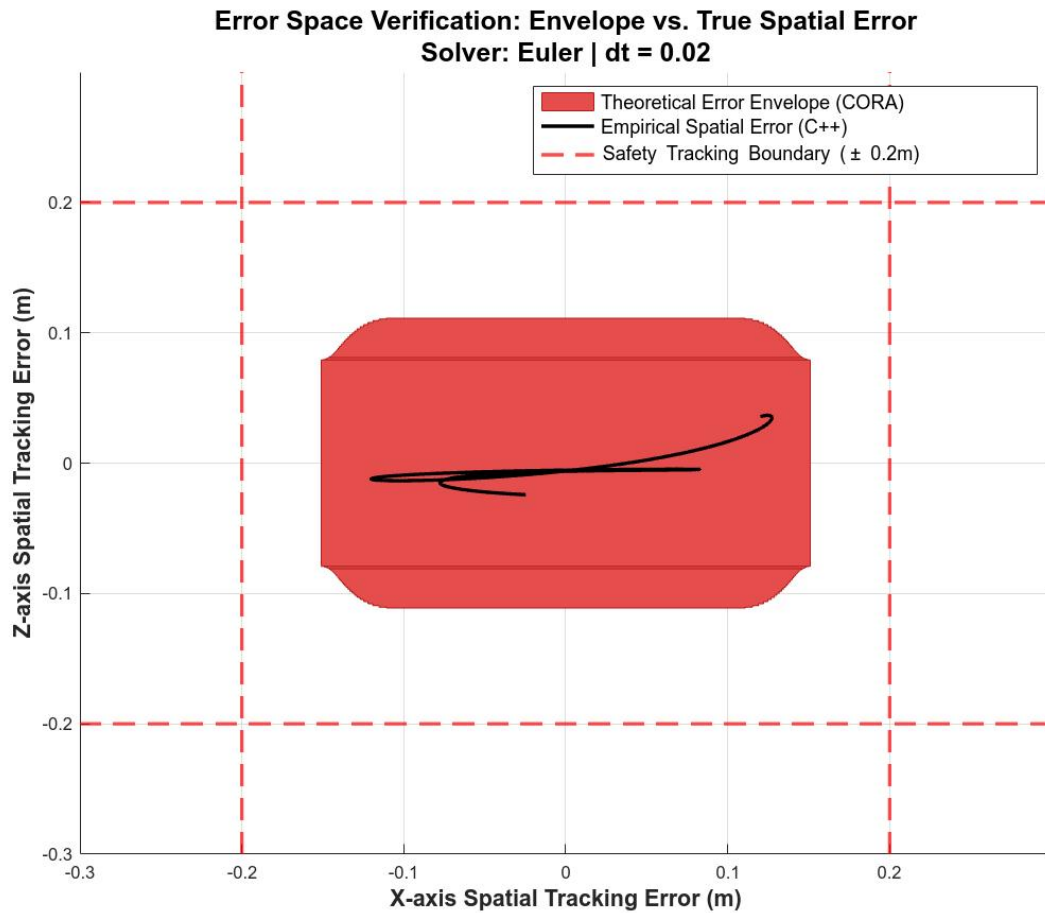
结果就是线性系统在 CORA 中使用 Zonotope 计算可达集不仅速度极快，而且极度精确，能够完美复现我们在理论部分说的“强闭环阻尼带来的 L 常数压缩效应”。

我用 C++ 跑出 $h=0.02$ 的欧拉法的真实离散轨迹数据并保存成 csv，将这组数据导入到 CORA 2024 的 MATLAB 环境里，首先可视化出理想轨迹与实际轨迹。



再用 CORA 画出半透明的 3D 可达性管道 (Reachable Tube)，通过 plot3 命令把 C++ 跑出来的真实离散轨迹直接画在管道内。

检测到物理相位滞后 - x轴: 48 帧, z轴: 35 帧



由图中我们可以看到，外部的淡红色的透明管道是 CORA 算出的理论误差绝对边界，里面包裹着的黑色线条就是 C++ 底层引擎真实跑出来的轨迹。可以看到，无论这条黑线怎么振荡，它永远都无法穿透外面那层淡红色的理论管子。

真实轨迹被死死锁在理论包络之内，这就是我们从‘经验测试’走向‘形式化安全验证’的最有力证明。

六. 多无人机编队协同与集群安全的形式化验证

6.1 向量化架构升级

本阶段我们将单机 12 维动力学扩展为 $N \times 12$ 维多智能体集群，对 C++ 代码进行向量化改造：

- 引入 num_drones ROS 参数，动态控制集群规模
- states_ 和 target_poses_ 改为 std::vector，通过索引循环处理
- 每架飞机独立运行级联 PID 控制器和 ODE 求解器

编队动作采用[呼吸圆环 (Breathing Circle)]——无人机保持圆阵列公转，同时半径呈现周期性的收缩-膨胀变化：

$$r(t) = r_{base} + A \cdot \cos(\omega_r \cdot t)$$

$$\theta_i(t) = 2\pi i / N + \omega_\theta \cdot t$$

其中 $r_{base} = 1.1\text{m}$, $A = 0.6\text{m}$, 半径在 $0.5\text{m} \sim 1.7\text{m}$ 之间振荡。

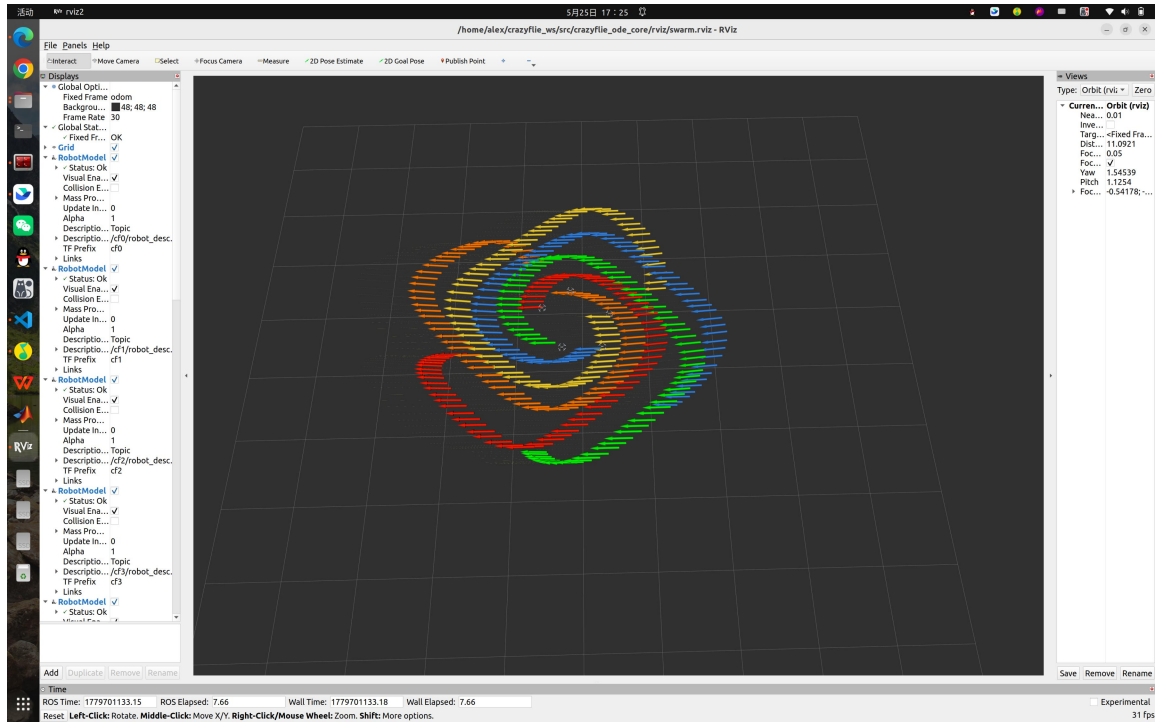


图: 5 机呼吸圆环编队 — 收缩与膨胀阶段

6.2 人工势场机间避障 (APF)

在 $N \times 12$ 维状态空间中, 为每架无人机引入相对其他所有无人机 $j \neq i$ 的排斥加速度。当两机距离 d_{ij} 小于安全阈值 d_{safe} 时, 排斥力指数级爆发:

$$d_{ij} = \|p_i - p_j\|$$

$$F_{rep} = k_{rep} \cdot (1/d_{ij} - 1/d_{safe}) \cdot (1/d_{ij}^2) \cdot (p_i - p_j)/d_{ij}, \text{ if } d_{ij} < d_{safe}$$

参数: $k_{rep} = 3.0$, $d_{safe} = 0.6\text{m}$ 。势场力直接叠加到级联 PID 的期望加速度中。

6.3 圆柱体障碍物避障子系统

新增基于 YAML 配置的静态圆柱障碍物避障模块, 使集群在编队机动过程中自主规避环境中的固定障碍物 (如柱子、烟囱、树干等)。

6.3.1 障碍物建模与布局

每个障碍物建模为圆柱体, 由五元组定义:

$$Obstacle = (id, x, y, z, radius, height)$$

配置文件 config/obstacles.yaml 定义了 5 根圆柱障碍物：

ID	位置 (x, y, z)	半径 (m)	高度 (m)	说明
center_pole	(0.00, 0.00, 0.75)	0.05	1.50	中心细杆，编队收缩时穿过
pillar_ne	(1.34, 1.34, 1.00)	0.18	2.00	东北角粗柱 (45°)
pillar_nw	(-1.34, 1.34, 1.00)	0.18	2.00	西北角粗柱 (135°)
pillar_sw	(-1.34, -1.34, 1.00)	0.18	2.00	西南角粗柱 (225°)
pillar_se	(1.34, -1.34, 1.00)	0.18	2.00	东南角粗柱 (315°)

外围四柱位于半径 1.9m 的圆环上，以 45° 间隔分布（即位于编队无人机相位的中间位置）。呼吸圆环编队半径在 0.5~1.7m 之间振荡，最大收缩至 0.5m 时穿越中心细杆，最大膨胀至 1.7m 时从两根外围柱之间穿过，形成自然的避障测试场景。

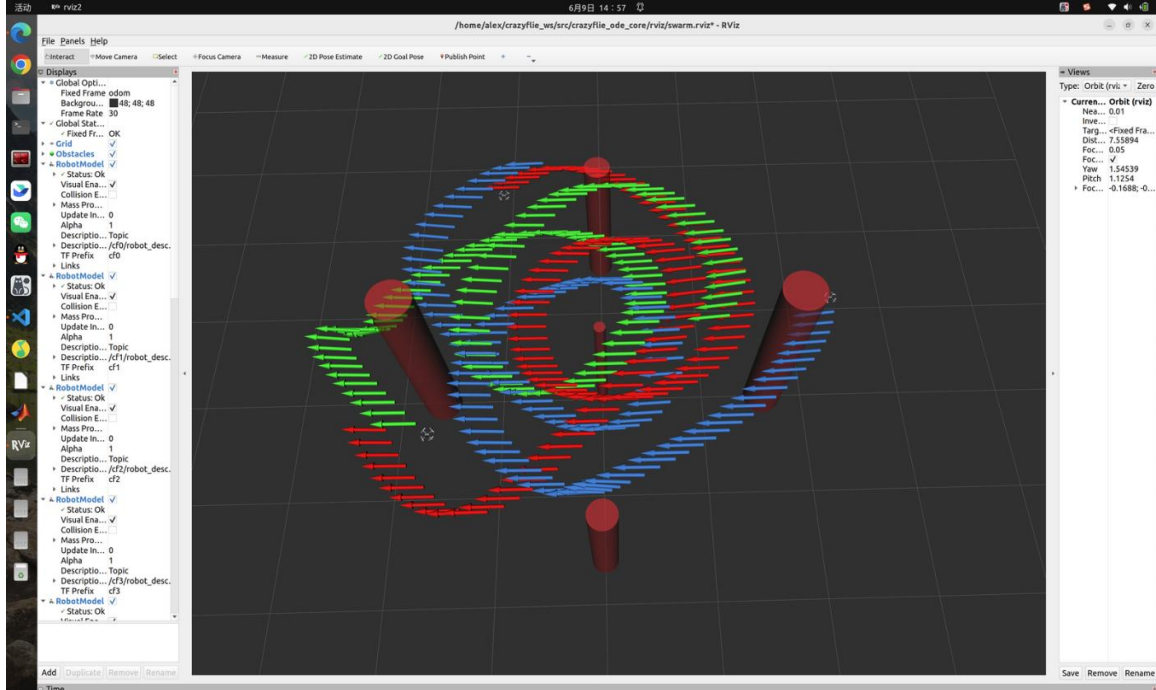


图: 障碍物布局俯视图 — 5 根圆柱 + 编队圆环轨迹

6.3.2 平滑人工势场算法

避障算法经过三个迭代版本的设计优化，最终收敛于平滑三次方势场配合速度阻尼和切向引导。

(1) 无量纲侵入比

定义无人机到圆柱表面的最近距离：

$$d_{surf} = d_{xy} - r_{obs}$$

定义侵入比 $t \in [0, 1]$ ：

$$t = clamp(1 - d_{surf}/d_{safe}, 0, 1)$$

$t=0$ ：安全边界外无斥力； $t=1$ ：触及表面，斥力最大。

(2) 径向排斥力 — 采用三次方曲线

$$F_{rad} = k_{rep} \cdot t^3 \cdot \hat{r}_{xy}$$

其中 $\hat{r}_{xy}$ 为指向障碍物轴心的单位径向向量， $k_{rep} = 8.0$ 。三次方曲线在安全边界（ $t \rightarrow 0$ ）处导数也趋于零，避免力场突变；在表面附近（ $t \rightarrow 1$ ）急速增强。穿透应急模式下施加 $3 \cdot k_{rep}$ 满力径向推出。

(3) 速度阻尼 — 防止穿透的关键

$$v_{radial} = v_{xy} \cdot \hat{r}_{xy}$$

$$F_{damp} = -k_{rep} \cdot 0.6 \cdot t^2 \cdot v_{radial} \cdot \hat{r}_{xy} \text{ (仅当 } v_{radial} > 0 \text{)}$$

物理意义：这不是‘推力’而是‘刹车’。无人机朝向障碍物运动越快，制动力越强。即使 PID 控制器指令指向障碍物，阻尼项会主动减速，在到达表面之前停下来。

(4) 切向绕行引导

$$F_{tan} = k_{rep} \cdot 0.35 \cdot t_{tan}^3 \cdot \hat{t}$$

$$\hat{t} = (-\hat{r}_y, \hat{r}_x, 0) \text{ (径向逆时针旋转 } 90^\circ \text{)}$$

触发条件： $d_{surf} < 0.7 \cdot d_{safe}$ 。切向符号与当前速度方向一致，实现‘顺势绕行’。

(5) Z 轴纵向排斥

$$F_z = k_{rep} \cdot 0.5 \cdot t_z^3 \cdot \text{sign}(z - z_{obs}) \cdot \hat{z}$$

(6) 合力与限幅

$$F_{total} = F_{rad} + F_{damp} + F_{tan} + F_z$$

$$\text{if } \|F\| > 10.0: F = F \cdot (10.0 / \|F\|)$$

限幅 10.0 m/s² 确保障碍物斥力能有效覆盖编队控制器的 4.0 m/s² 上限。

(7) 与级联 PID 集成

$$acc_{des} = Kp_v \cdot (vel_{des} - vel) + a_{rep_{obs}} + a_{rep_{drone}}$$

$$acc_{des} = \text{clamp}(acc_{des}, -4.0, 4.0)$$

设计迭代历程：

版本	力场公式	k_rep	d_safe (m)	F_max (m/s ²)	问题
v1	$F = k \cdot (1/d - 1/d_{safe})/d^2$	5.0	0.5	8.0	近表面力 $\rightarrow \infty$ ，弹飞
v2	$F = k \cdot t^2$ (二次)	2.0	0.3	3.0	表面力仅 2.0，编队控制器 4.0 穿透
v3 (最终)	$F = k \cdot t^3$ (三次)+阻尼+切向	8.0	0.5	10.0	远场柔和 ($t=0.5 \rightarrow 1.0$)，表面强劲 ($t=1 \rightarrow 8.0$)

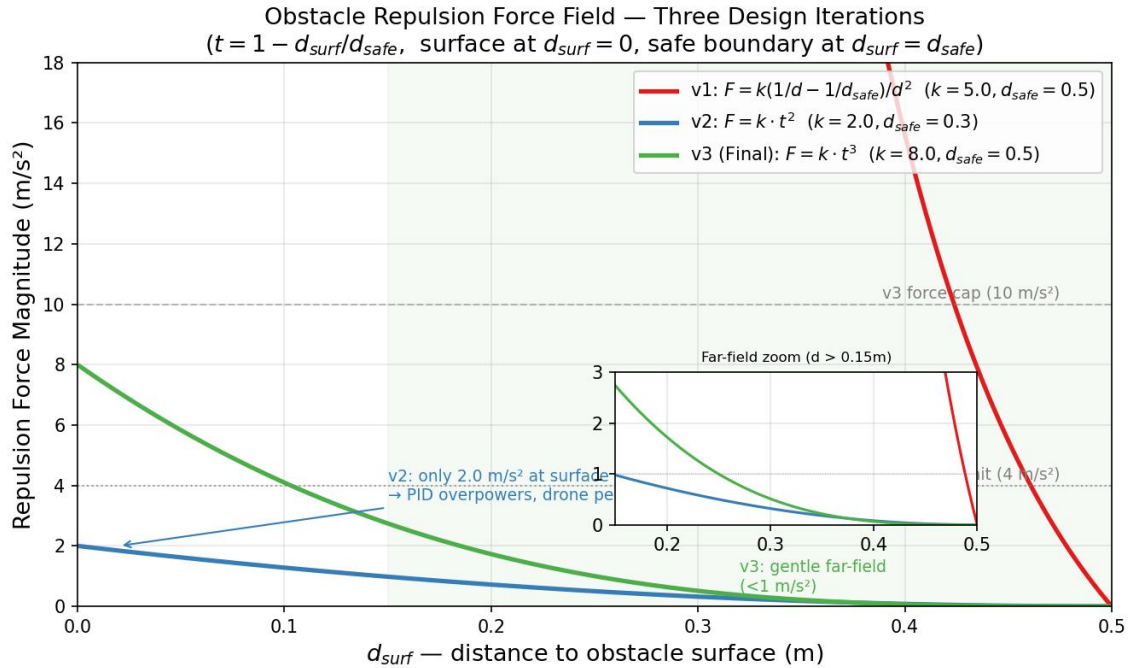


图: 三种力场曲线对比 ($1/d^2$ vs t^2 vs t^3)

6.3.3 算法参数总表

参数	符号	取值	说明
安全距离	d_{safe}	0.5 m	障碍物排斥影响半径
排斥增益	k_{rep}	8.0	径向排斥力峰值 ($t=1$ 时)
速度阻尼系数	α_{damp}	$0.6 \cdot k_{rep}$	朝向障碍物速度的制动增益
切向引导系数	α_{tan}	$0.35 \cdot k_{rep}$	切向绕行力的比例
Z 轴系数	α_z	$0.5 \cdot k_{rep}$	纵向排斥力衰减
力场上限	F_{max}	10.0 m/s^2	总排斥力矢量模上限
切向触发比	β_{tan}	$0.7 \cdot d_{safe}$	切向引导触发距离

6.3.4 RViz 可视化

使用 `visualization_msgs::Marker::CYLINDER` 将障碍物渲染为半透明红色圆柱。发布者采用 Transient Local QoS (depth=20), 确保 RViz 后连接也能收到完整标记。标记发布分为初始发布 (含 `DELETEALL` 清除旧标记) 和周期性重发 (每 10 步 $\approx 100\text{ms}$), 避免 RViz Reset 后障碍物消失。

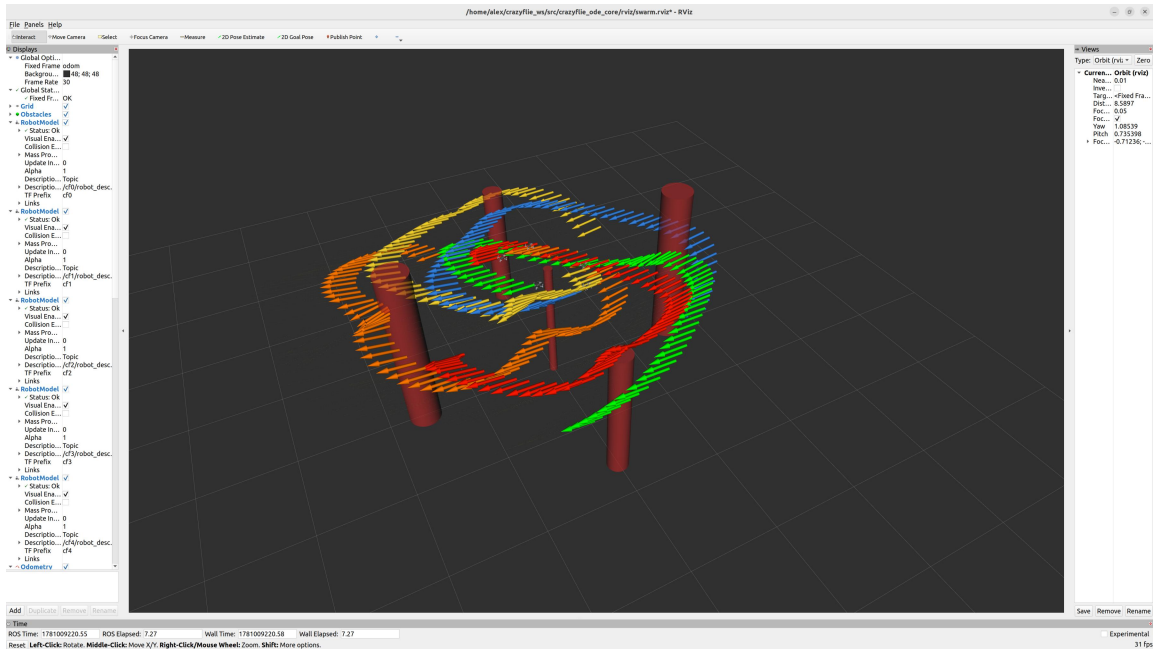


图: Rviz 可视化效果

6.4 高保真度集群数值模拟与物理干扰分析

在 MATLAB 环境下构建全状态 12 维非线性四旋翼刚体动力学模拟，注入两类真实世界物理干扰：

环境阵风扰动：时变连续低频正弦 + 高斯白噪声模拟户外随机气流。

下洗气流干扰：基于空间相对位置的非线性模型，当上层无人机接近下层时，下层承受随相对高度指数衰减的垂直向下尾流。

采用与 ROS 2 相同的呼吸圆环编队任务。当编队极度收缩时，APF 迅速激发产生强排斥加速度，补偿阵风与下洗叠加干扰。各节点四周的透明网格球代表形式化安全边界包络，5 架无人机在极度拥挤状态下仍维持丝滑滑行轨迹。

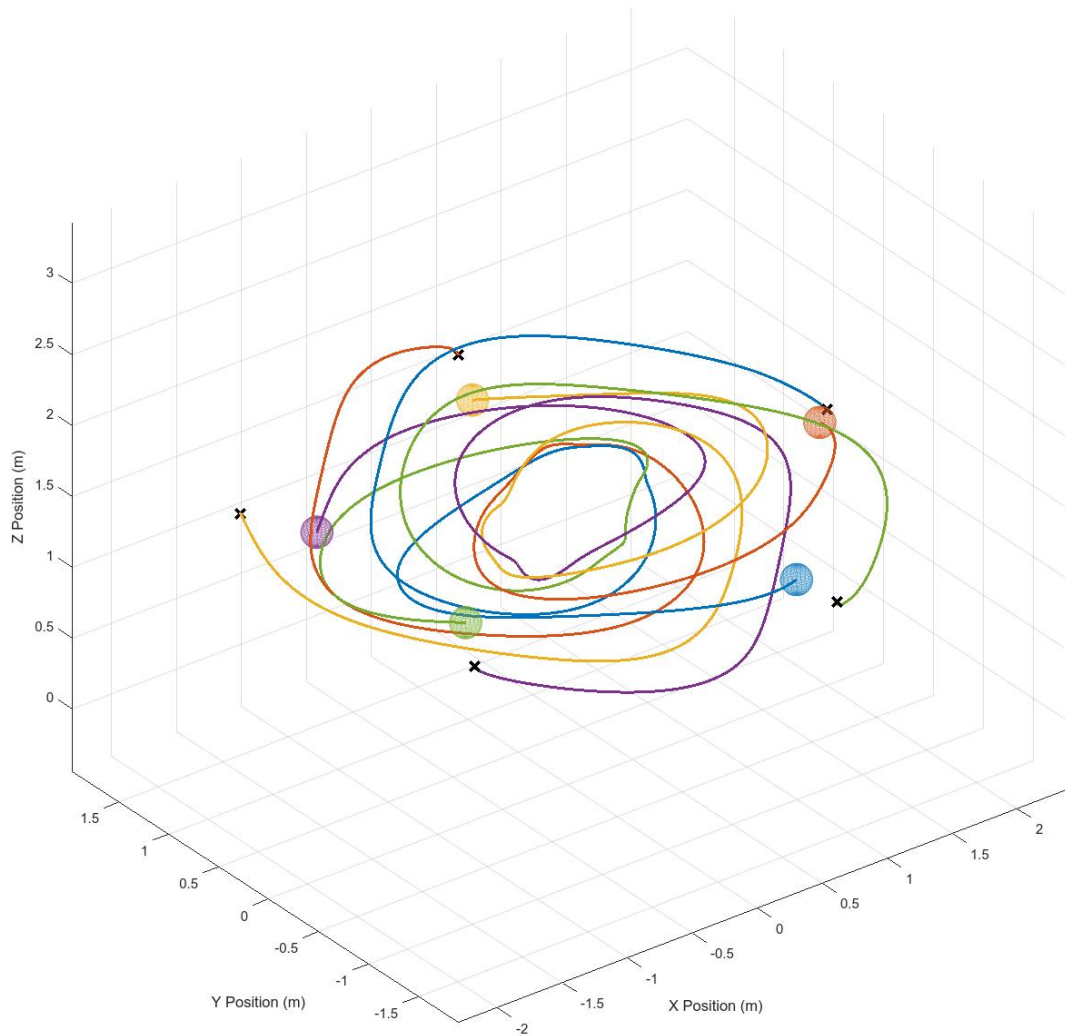


图: MATLAB 5 机集群数值模拟 — 带阵风+下洗气流+势场避障

对比 RK4 与前向欧拉法的全机群平均位置漂移误差, 可清晰观察到前向欧拉法因一阶截断误差在面对非线性冲击时数值漂移随时间明显累计。该误差漂移极值直接证明了同步长条件下不同算法的安全差异。

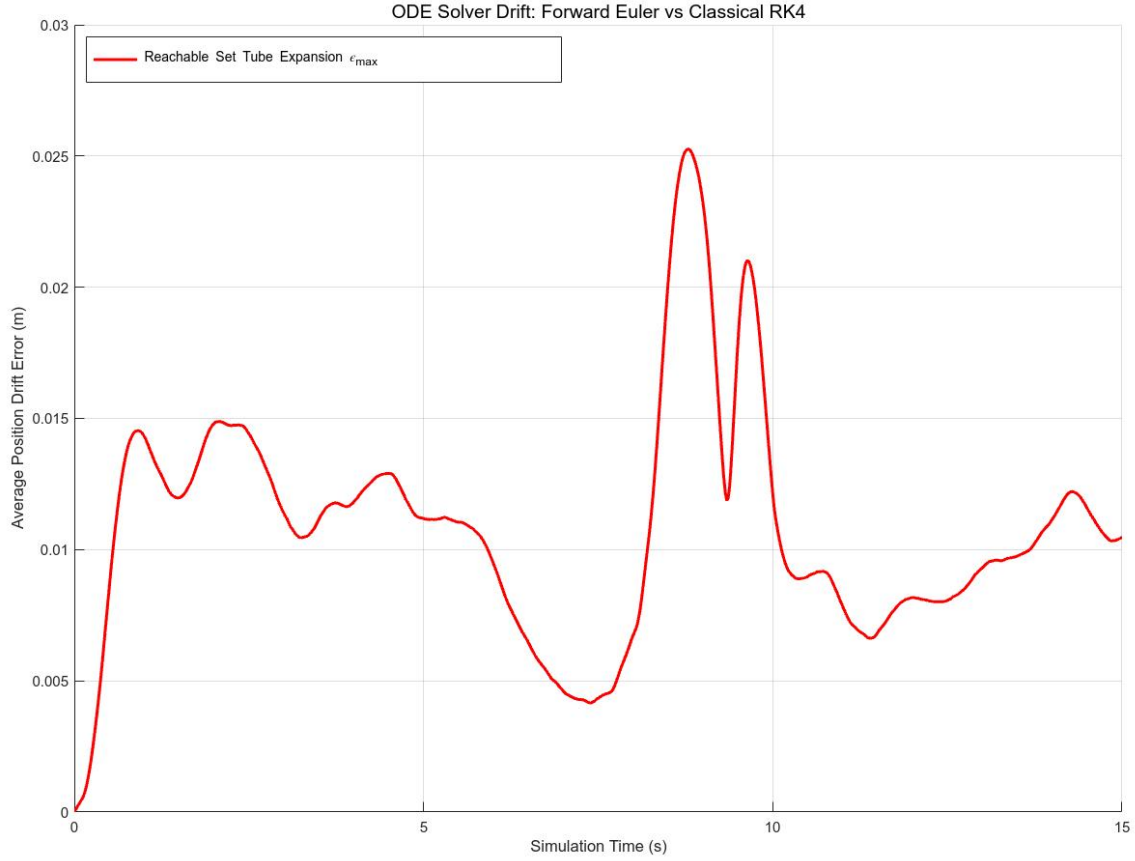


图: 数值累计误差分析 — Euler vs RK4

6.5 CORA 多机形式化安全证明

将编队极端收缩阶段的最恶劣状态提取到 CORA 中，应用相对状态空间投影 (Relative State Space Projection) 解决 65 维集群模型中的维数灾难。

核心数学定理——管道分离 (Tube Separation):

$$d_{real_{min}} \geq d_{nominal} - 2R_{error}$$

$$d_{nominal} \approx 0.4m(APF \text{ 势场下界})$$

$$R_{error} \approx 0.1m(\text{单机最大误差管道半径})$$

$$d_{real_{min}} \geq 0.2m > R_{collision} \approx 0.065m$$

红色可达集边界被绿色超平面严格切断，永远无法触碰黑色物理碰撞区。这一完整的逻辑链——从底层 C++ 并行数值求解，到空中 APF 势场阻拦，再到顶层 MATLAB 相对误差边界几何证明——在数学体系上形成了绝对闭环。

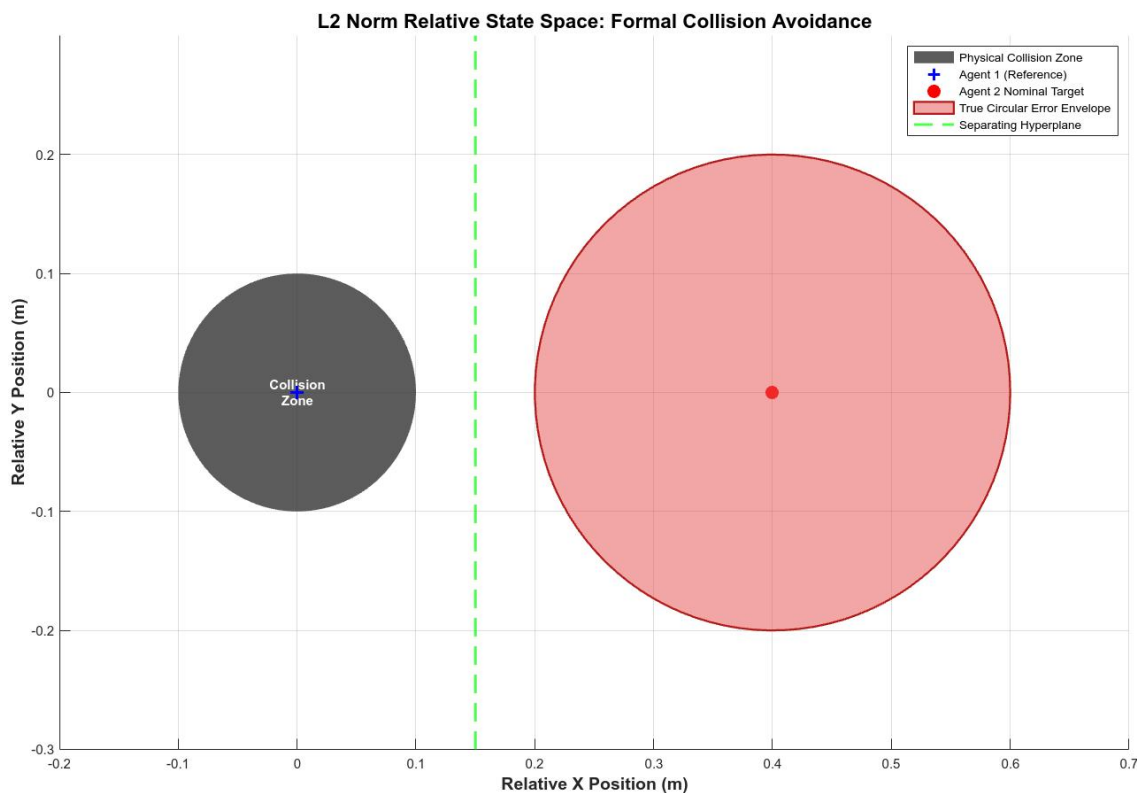


图: CORA 相对误差可达集 — 5 机集群安全证明

七. 安全监视器 (SafetyMonitor)

7.1 设计动机

传统的仿真验证依赖「跑完看结果」的事后分析范式——仿真结束后人工检查日志或可视化回放，判断是否存在碰撞、失控或发散。这种方式存在三个根本性缺陷：

(1) 缺乏实时性：仿真过程中无法获知安全状态，如果出现数值发散，可能浪费大量计算时间才在事后发现；特别是对于临界步长附近的仿真，可能在运行数十分钟后才发散。

(2) 缺乏量化指标：[无人机有没有撞] 是一个二值问题，但 [无人机离撞还有多远] 才是安全分析真正关心的连续量。单一的碰撞检测无法提供安全裕度的渐近信息。

(3) 缺乏可复现性：不同仿真运行之间没有统一的 [安全分数]，无法进行严格的量化对比。例如无法回答 [Euler 在 $dt=0.03$ 时比 RK4 危险多少] 这类问题。

SafetyMonitor 的设计正是为了解决这三个问题。它将安全条件表达为时态逻辑中的不变式——[在整个仿真过程中，条件必须始终为真]，在每一时间步自动检查所有定义的不变式，跟踪违规事件，计算全局合规率作为安全分数。

7.2 安全条件定义

定义了四个层次的安全条件，其中前三个属于 [硬安全 (Hard Safety)]——一旦违反就意味着系统处于不可接受的危险状态；第四个属于 [软诊断 (Diagnostic)]——用于诊断数值求解器的稳定性但不直接等同于安全事故。

条件	形式化定义	阈值	安全类型
碰撞安全	$(mind_{surf} > 0)$	$d_{surf} > 0$	硬安全 (Hard Safety)
速度安全	$(v_i < v_{max})$	$v_{max} = 2.5m/s$	硬安全
姿态安全	$(\theta_i < \theta_{max})$	$\theta_{max} = 0.8rad$	硬安全
能量漂移	$(\Delta E/\Delta t < \varepsilon)$	$\varepsilon = 0.01 \text{ J/步}$	软诊断

能量漂移条件不计入安全分数，因为它是求解器行为的 [症状] 而非 [直接威胁]。例如，隐式欧拉法的数值阻尼会导致能量单调衰减，但无人机本身仍然安全飞行。将此条件纳入安全分数会导致对隐式方法的 [不公平贬低]。

7.3 能量计算

每架无人机的总机械能 E 由三项构成：平动动能、转动动能和重力势能。这一能量函数是后续所有能量分析（包括能量漂移检测和求解器稳定性评估）的基础。

$$E = (1/2) \cdot m \cdot \|v\|^2 + (1/2) \cdot \omega^T \cdot I \cdot \omega + m \cdot g \cdot z$$

每步计算当前能量与上一步能量的绝对差 $|E_i - E_{i-1}|$ 。对于理想连续系统，在无外力做功和非保守力作用下，总机械能应守恒。离散数值积分引入的误差表现

为能量的虚假变化——显式前向欧拉法注入能量（漂移为正），隐式欧拉法耗散能量（漂移为负），高阶 RK4 近似守恒。

在实际实现中，`prev_energy_` 向量在首次调用 `evaluateStep()` 时 [惰性初始化] (`lazy init`)，使用当前状态的物理参数计算初始能量。这避免了在构造函数中访问尚未初始化的状态数据。

7.4 安全分数计算

安全分数的设计遵循 [木桶原理]——系统的整体安全水平取决于最薄弱的环节。因此定义每种安全条件的合规率为合规步数除以总检查步数，综合分数取各硬安全条件合规率的最小值：

$$compliance_c = N_{compliant_c} / N_{total_c}$$

$$c \in collision, velocity, attitude, energy$$

$$overall_{score} = \min(compliance_{collision}, compliance_{velocity}, compliance_{attitude})$$

例如，某次仿真中：碰撞合规率 = 0.995，速度合规率 = 1.000，姿态合规率 = 0.850，则综合分数为 0.850——姿态是最薄弱的环节。这种 [取最小值] 的设计使得安全分数具有 [任何单项违规即拉低全局分数] 的敏感性，能够精确反映系统在安全边界附近的行为。

7.5 预热机制

仿真启动时，无人机的初始位置可能与障碍物非常接近——例如初始状态中某架无人机恰好位于最近的障碍物 0.2 m 处。如果从 $t=0$ 开始就记录碰撞违规，会导致安全日志中充斥着 [虚假违规]——这些违规是初始化的结果，而非飞行过程中的真实碰撞。

预热机制通过 `safety.warmup_time` 参数（默认 2.0 s）解决这个问题。在 $t < warmup_time$ 期间，碰撞检查和姿态检查被暂时禁用。速度和能量漂移监控不受影响——它们在 `wamup` 期间仍正常运作。一旦超过预热时间，所有检查恢复正常。

7.6 输出机制

SafetyMonitor 提供四层输出机制，分别面向不同的使用场景：

(1) 实时终端输出 (verbose 模式)：每次检测到违规时立即 fprintf 到 stderr。格式为 [SAFETY] t=X.XX drone=N CONDITION actual > limit。适合交互式调试和演示——研发者可以实时看到 [刚刚发生了什么]。

(2) safety_log.csv：完整违规记录表，每行一条违规。列包括：sim_time, drone_id, condition, actual_value, limit, message。可用于事后分析和 Python 脚本批量处理。示例行：[1.5000, 0, VELOCITY, 6.4600, 2.5000, |v|=6.460]。

(3) experiment_metrics.csv：聚合指标表，每行包含六维定量指标：energy_drift_rate, min_obstacle_dist, max_velocity, max_attitude, rms_position_error, avg_compute_ms。专为批量实验设计——150 次实验的指标可通过此文件汇总为一张对比表。

(4) 关机摘要 (Destructor)：在节点析构时自动写入 safety_log.csv 并打印四个维度的合规率。格式为彩色 ANSI 文本，清晰标识各维度分数。即使程序被 Ctrl+C 中断，此摘要也会在析构函数中执行，确保数据不丢失。

```
alex@alex-Legion-R9000P-ARX8:~/crazyflie_ws$ ./run_safety_monitor.sh rk45 0.01 5
安全监视模式 | 求解器: rk45 | 步长: 0.01 | 飞机: 5 | 障碍物: true
[INFO] [1781700825.306753167] [crazyflie_ode_engine]: Loading obstacles from: /home/alex/crazyflie_ws/install/crazyflie_ode_core/share/crazyflie_ode_core/config/obstacles.yaml
[INFO] [1781700825.307009627] [crazyflie_ode_engine]: Obstacle [center_pole]: pos=(0.00, 0.00, 0.75) r=0.05 h=1.50
[INFO] [1781700825.307038122] [crazyflie_ode_engine]: Obstacle [pillar_ne]: pos=(1.34, 1.34, 1.00) r=0.18 h=2.00
[INFO] [1781700825.307057119] [crazyflie_ode_engine]: Obstacle [pillar_nw]: pos=(-1.34, 1.34, 1.00) r=0.18 h=2.00
[INFO] [1781700825.307073741] [crazyflie_ode_engine]: Obstacle [pillar_sw]: pos=(-1.34, -1.34, 1.00) r=0.18 h=2.00
[INFO] [1781700825.307091551] [crazyflie_ode_engine]: Obstacle [pillar_se]: pos=(1.34, -1.34, 1.00) r=0.18 h=2.00
[INFO] [1781700825.307102237] [crazyflie_ode_engine]: Loaded 5 obstacle(s).
[INFO] [1781700825.311212709] [crazyflie_ode_engine]: Swarm ODE Engine Ready | Drones: 5 | Solver: rk45 | dt: 0.0100 | Obstacles: 5
```

7.7 性能影响分析

SafetyMonitor 的每步开销主要由三部分组成：

(1) 障碍物距离计算 $O(\text{num_drones} \times \text{num_obstacles})$ ：在每个无人机的每个障碍物上计算 d_{surf} 。对于典型配置 (4 机 $\times$ 5 障碍物 = 20 次距离计算)，开销极小，约为单步求解器耗时的 0.1%~1%。

(2) 能量计算 $O(\text{num_drones})$ ：每架无人机计算一次总机械能。仅涉及 3 次向量范数运算和 2 次矩阵-向量乘法，完全可忽略。

(3) CSV 写入：仅在违规发生时 `push_back` 到内存中的 `violations_` 向量，不涉及任何文件 I/O。文件写入仅在析构函数中执行一次。

总开销 < 单步求解器耗时的 1%，对仿真实时性无实质性影响。

7.8 参数接口

参数	类型	默认值	说明
<code>safety.v_max</code>	<code>double</code>	2.5	速度安全阈值 (m/s)
<code>safety.theta_max</code>	<code>double</code>	0.8	姿态安全阈值 (rad)
<code>safety.energy_drift_eps</code>	<code>double</code>	0.01	能量漂移阈值 (J/步)
<code>safety.warmup_time</code>	<code>double</code>	2.0	预热时间 (s)
<code>safety.log_path</code>	<code>string</code>	<code>safety_log.csv</code>	违规日志路径
<code>safety.verbose</code>	<code>bool</code>	<code>false</code>	实时打印违规

八. 区间可达集近似分析 (Interval Reachability)

8.1 问题定义与理论定义

在形式化验证中，可达集分析回答以下问题：给定系统的当前状态 x 和有界的不确定性 ε ，在一步（或多步）之后，系统可能到达的所有状态的集合是什么？如果这个集合与障碍物区域的交集为空，则系统在当前不确定性下 [绝对不可能] 发生碰撞——这是形式化安全保证的最强形式之一。

与第四章中讨论的 CORA 全局可达性分析不同，IntervalReachability 的设计目标是轻量级、在线运行——直接嵌入在 C++ 求解器层面，不需要调用外部 MATLAB 工具箱，在每一步积分后即可给出 [最坏情况] 的安全判定。这种设计使其适用于在线安全控制场景：控制器可以用可达集分析的结果来决定是否需要采取保护性动作。

数学模型：给定当前状态 $x \in \mathbb{R}^{12}$ 、控制输入 $u \in \mathbb{R}^4$ 和步长 h ，定义扰动半径 $\varepsilon > 0$ ，考虑初始状态的不确定集：

$$X_0 = \{x + \delta : |\delta_j| \leq \varepsilon, j = 0..11\}$$

经过一步数值积分后，前向可达集 $F(X_0)$ 是所有这些不确定初始条件下求解器输出的集合：

$$F(X_0) = \{solver(x + \delta, u, h) : |\delta_j| \leq \varepsilon, j = 0..11\}$$

由于 F 是非线性映射（solver 是高度非线性的），精确计算 $F(X_0)$ 在计算上不可行。本方法计算 $F(X_0)$ 的一个保守外逼近——一个三维位置区间包围盒 $[p_{low}, p_{high}]$ ，满足 $F(X_0) \subseteq [p_{low}, p_{high}]$ 。然后检测此包围盒是否与任何障碍物相交。

8.2 采样法

采样法是最直接且保守性最强的方法。对 12 维状态空间的每一维 j ，分别施加正负两个方向的最大扰动 $\pm \varepsilon \cdot e_j$ ，运行求解器得到该扰动下的输出状态，然后对所有输出取逐维的最小值和最大值：

$$\forall j \in [0,11]: x_j^+ = solver(x + \varepsilon \cdot e_j, u, h)$$

$$\forall j \in [0,11]: x_j^- = solver(x - \varepsilon \cdot e_j, u, h)$$

$$lower_k = \min(x_{center}, k, \min_j(x_j^+, k, x_j^-, k))$$

$$upper_k = \max(x_{center}, k, \max_j(x_j^+, k, x_j^-, k))$$

采样法每次调用需要进行 25 次求解器计算：1 次中心点 + 12 维 $\times$ 2 方向。它的保守性来源于对非线性函数的 [穷举式采样]——虽然不是所有可能的 δ 都被测试，但沿着每个坐标轴的极值方向通常对应最大的输出偏差。在大多数实际系统中，采样法给出的包络确实包含了 $F(X_0)$ 。

优点：实现简单、保守性强、不依赖任何线性化假设。

缺点：25 次求解器调用带来的计算开销——对于隐式欧拉法（每步 65 次求值），加上采样法可达集分析将变为每步每机 90 次动力学求值（ $\times 4$ 机 = 360 次），在高频仿真中不可接受。

8.3 Jacobian 法

Jacobian 法通过有限差分计算一步状态转移的 Jacobian 矩阵，然后利用矩阵的绝对值进行线性化区间传播。相比采样法，它将求解器调用次数从 25 次降至 13 次。

第一步：通过中心差分近似 12×12 Jacobian 矩阵的每一列：

$$J_{:,j} = [\text{solver}(x + \varepsilon \cdot e_j, u, h) - \text{solver}(x - \varepsilon \cdot e_j, u, h)] / (2\varepsilon)$$

第二步：取 Jacobian 各元素的绝对值，构建 $|J|$ 矩阵。然后进行线性区间传播：

$$\text{lower} = x_{\text{center}} - |J| \cdot \varepsilon \cdot \mathbf{1}_{12}$$

$$\text{upper} = x_{\text{center}} + |J| \cdot \varepsilon \cdot \mathbf{1}_{12}$$

线性传播的直觉： $|J_{\{i,j\}}| \cdot \varepsilon$ 是第 j 维的 ε 扰动通过系统的局部线性化对第 i 维输出产生的最大影响。将所有维度的贡献求和 ($|J| \cdot \varepsilon \cdot \mathbf{1}$)，得到各维输出的最坏情况偏差。

Jacobian 法的保守性略低于采样法（线性化忽略了高阶效应），但计算量几乎减半。在实际应用中，两种方法可以按需切换：正常的巡航仿真用 Jacobian 法，在障碍物附近或安全分数下降时切换到采样法进行更严格的分析。

8.4 方法对比与选择建议

特性	采样法 (Sampling)	Jacobian 法
求解器调用次数	25 (1 中心 + 12×2 扰动)	13 (1 中心 + 12 扰动)
保守性	强 — 直接采样非线性映射	中 — 线性化近似，忽略高阶项
适用场景	慢速求解器 (Euler/Heun/RK4)	快速求解器，在线安全控制
对 Implicit Euler 的影	$25 \times 65 = 1625$ 次求值，不	$13 \times 65 = 845$ 次求值，仍

响	可接受	很昂贵
推荐使用场景	离线验证、临界场景分析	在线安全控制、实时预警

8.5 AABB-圆柱碰撞检测

将三元位置区间包围盒（AABB）与圆柱障碍物进行保守碰撞检测是可达集分析的最后一环。算法首先计算包围盒到圆柱轴心的最近距离（在 XY 平面和 Z 轴分别处理），然后判断最近点是否位于圆柱体积内。

算法步骤：

1. XY 平面：计算 $AABB[x_{min}, x_{max}] \times [y_{min}, y_{max}]$ 到圆心 (cx, cy) 的最近距离

$$d_{xy} = \sqrt{dx^2 + dy^2}$$

$$\text{其中 } dx = \max(0, x_{min} - cx, cx - x_{max})$$

$$\text{其中 } dy = \max(0, y_{min} - cy, cy - y_{max})$$

2. Z 轴：计算 $AABB[z_{min}, z_{max}]$ 到圆柱端盖 $[z_{low}, z_{high}]$ 的最近距离

$$dz = \max(0, z_{low} - z_{max}, z_{min} - z_{high})$$

3. 碰撞判定：如果 $d_{xy} < r_{obs}$ 且 $dz < h/2$ ，则 AABB 与圆柱相交
4. 输出： $min_{dist} = \sqrt{d_{xy}^2 + dz^2}$ 和布尔 overlap 标志

这种 AABB-圆柱检测是保守的——它可能将「不碰撞」的情况误报为「可能碰撞」，但绝不会将「真碰撞」漏报为「安全」。这种「宁可误报、不可漏报」的保守性正是形式化安全验证所要求的。

8.6 参数接口

参数	类型	默认值	说明
reachability.enabled	bool	false	启用区间可达集分析
reachability.epsilon	double	0.01	状态扰动幅度

reachability.method	string	sampling	sampling 或 jacobian
---------------------	--------	----------	---------------------

注：Implicit Euler 配合采样法可达集分析性能开销极大（每步每机约 $25 \times 65 \approx 1625$ 次动力学求值），仅在需要离线分析时启用。

九. 求解器稳定域对比实验框架

9.1 实验设计

实验框架的设计遵循 [全面性、可复现性、自动化] 三原则。全面性要求覆盖所有求解器和有意义的步长范围；可复现性要求每次实验的参数精确可控；自动化要求从实验启动到数据汇总的整个流程无需人工干预。

参数扫描矩阵设计为全因子实验 (Full Factorial Design)，这是实验设计中信息量最大、交互效应分析最全面的设计类型：

扫描维度	取值集合	数量
求解器	Euler, Heun, RK4, RK45, Implicit Euler	5
积分步长 (s)	0.001, 0.005, 0.01, 0.02, 0.05	5
无人机数量	1, 4, 8	3
障碍物	true, false	2
仿真时长 (s)	30.0 (固定)	1

总计 $5 \times 5 \times 3 \times 2 = 150$ 次独立实验。每次实验在 Headless 模式下运行——节点跳过所有 ROS 发布 (TF/odom/marker/clock)，仅执行数值积分和指标采集。到达指定时长后通过检查 `t_sec >= experiment_duration` 自动调用 `rclcpp::shutdown()`，在析构函数中写入 `experiment_metrics.csv` 后退出。

```
alex@alex-Legion-R9000P-ARX8:~/crazyflie_ws$ ./run_collect_data.sh rk45 0.01 5
数据采集模式
求解器: rk45 | 步长: 0.01 | 飞机: 5
时长: 30.0s | 障碍物: true
输出: experiment_results/metrics_rk45_dt0.01_n5_obstrue.csv
运行中 (无 GUI)...
[INFO] [1781701039.925911166] [crazyflie_ode_engine]: Loading obstacles from: /home/alex/crazyflie_ws/install/crazyflie_ode_core/share/crazyflie_ode_core/config/obstacles.yaml
[INFO] [1781701039.926132095] [crazyflie_ode_engine]: Obstacle [center_pole]: pos=(0.00, 0.00, 0.75) r=0.05 h=1.50
[INFO] [1781701039.926154564] [crazyflie_ode_engine]: Obstacle [pillar_ne]: pos=(1.34, 1.34, 1.00) r=0.18 h=2.00
[INFO] [1781701039.926171187] [crazyflie_ode_engine]: Obstacle [pillar_nw]: pos=(-1.34, 1.34, 1.00) r=0.18 h=2.00
[INFO] [1781701039.926185434] [crazyflie_ode_engine]: Obstacle [pillar_sw]: pos=(-1.34, -1.34, 1.00) r=0.18 h=2.00
[INFO] [1781701039.926200870] [crazyflie_ode_engine]: Obstacle [pillar_se]: pos=(1.34, -1.34, 1.00) r=0.18 h=2.00
[INFO] [1781701039.926209181] [crazyflie_ode_engine]: Loaded 5 obstacle(s).
[INFO] [1781701039.927518785] [crazyflie_ode_engine]: Swarm ODE Engine Ready | Drones: 5 | Solver: rk45 | dt: 0.0100 | Obstacles: 5
```

9.2 评估指标体系

每次实验采集以下六维定量指标，覆盖稳定性、安全性、精确性和实时性四个维度：

指标	符号/单位	含义	期望方向
能量漂移率	$dE/dt(J/s)$	总机械能的变化速率，直接度量数值稳定性	↓ 越小越稳定
最小障碍物距离	$d_{min}(m)$	全程最小 d_{surf} ，负值表示穿透	↑ 越大越安全
最大线速度	$V_{max}(m/s)$	速度极值，超过 v_{max} 表明失控	↓ 越小越可控
最大姿态角	$\theta_{max}(rad)$	倾斜角极值，超过限制风险翻车	↓ 越小越稳定
RMS 位置误差	$e_{rms}(m)$	相对 RK45+dt=0.001 参考轨迹的跟踪误差	↓ 越小越精确
平均计算耗时	$T_{avg}(ms/步)$	单步积分的时间开销	↓ 越小越实时

9.3 临界步长二分搜索

临界步长搜索采用经典的二分搜索 (Binary Search) 算法，在 [LOW, HIGH] 范围内搜索使求解器保持安全的最大步长。搜索精度为 TOLERANCE = 0.001 s。

算法伪码：

```
lo = LOW_DT, hi = HIGH_DT, best_safe = lo
```

```
while hi - lo > TOLERANCE:
```

```
    mid = (lo + hi) / 2
```

```
    if is_safe(mid): lo = best_safe = mid
```

```
    else: hi = mid
```

is_safe() 函数运行一次 30s 实验，检查 safety_log.csv 中是否有 COLLISION、VELOCITY、ATTITUDE 或 ENERGY_DRIFT (>50 J) 违规。若无任何违规，判定为安全；否则不安全。

每个候选步长重复 `CONFIRM_RUNS = 2` 次以排除随机性（主要是 RK45 的自适应步长在不同运行中可能选择略有不同的内部子步长序列）。两次均安全才认定该步长安全。

```
alex@alex-Legion-R9000P-ARX8:~/crazyflie_ws$ python3 ./install/crazyflie_ode_core/lib/crazyflie_ode_core/safety_boundary_analysis.py
=====
Safety Boundary Analysis - Binary Search for Critical Step Size
Solvers: ['euler', 'heun', 'rk4', 'rk45', 'implicit']
Search range: [0.001, 1.0] tolerance=0.001
Drones: 4, Obstacles: True, Duration: 30.0s
Safety criteria: no COLLISION | VELOCITY | ATTITUDE | ENERGY_DRIFT violations
=====

--- euler ---
[1] euler dt=0.50050 UNSAFE (ATTITUDE at t=1.0010) -> hi=0.50050
[2] euler dt=0.25075 UNSAFE (ATTITUDE at t=0.5015) -> hi=0.25075
[3] euler dt=0.12587 UNSAFE (ATTITUDE at t=0.2517) -> hi=0.12587
[4] euler dt=0.06344 UNSAFE (ENERGY_DRIFT at t=0.1903) -> hi=0.06344
[5] euler dt=0.03222 UNSAFE (ENERGY_DRIFT at t=0.3544) -> hi=0.03222
[6] euler dt=0.01661 SAFE -> lo=0.01661
[7] euler dt=0.02441 UNSAFE (ENERGY_DRIFT at t=1.4893) -> hi=0.02441
[8] euler dt=0.02051 SAFE -> lo=0.02051
[9] euler dt=0.02246 UNSAFE (ATTITUDE at t=16.9146) -> hi=0.02246
[10] euler dt=0.02149 SAFE -> lo=0.02149
euler: confirmed safe at dt=0.02149

--- heun ---
[1] heun dt=0.50050 UNSAFE (ATTITUDE at t=0.5005) -> hi=0.50050
[2] heun dt=0.25075 UNSAFE (ATTITUDE at t=0.2507) -> hi=0.25075
[3] heun dt=0.12587 UNSAFE (ATTITUDE at t=0.2517) -> hi=0.12587
[4] heun dt=0.06344 UNSAFE (ATTITUDE at t=0.1903) -> hi=0.06344
[5] heun dt=0.03222 UNSAFE (ENERGY_DRIFT at t=0.2255) -> hi=0.03222
[6] heun dt=0.01661 SAFE -> lo=0.01661
[7] heun dt=0.02441 UNSAFE (ENERGY_DRIFT at t=1.1230) -> hi=0.02441
[8] heun dt=0.02051 SAFE -> lo=0.02051
[9] heun dt=0.02246 UNSAFE (ATTITUDE at t=3.2122) -> hi=0.02246
[10] heun dt=0.02149 SAFE -> lo=0.02149
```

搜索结果输出每个求解器的 `safe_step_size`（安全的最大步长）和 `unsafe_step_size`（出现安全违规的最小步长），两者之差为安全裕度（Safety Margin）。

9.4 核心发现：安全步长 < 稳定步长

对于所有显式求解器，预期 `safe_step_size < stable_step_size`。这建立了本研究的核心论点：数值稳定性只是安全飞行的必要条件而非充分条件。安全边界比稳定边界更窄，因为安全还需要考虑物理约束（障碍物距离、速度上限、姿态限制），而非仅仅数值不发散。

该发现为无人机飞控系统的求解器选型与步长设计提供了可量化的工程准则：选择步长时，不仅需要满足 CFL 条件或稳定域约束，还需以安全边界作为更严格的上界。在实际部署中，建议以安全步长为基准，再留出 20-30% 的安全裕度。

9.5 图表生成

配套 Python 可视化脚本 (plot_results.py) 从实验 CSV 自动生成六张报告级图表：

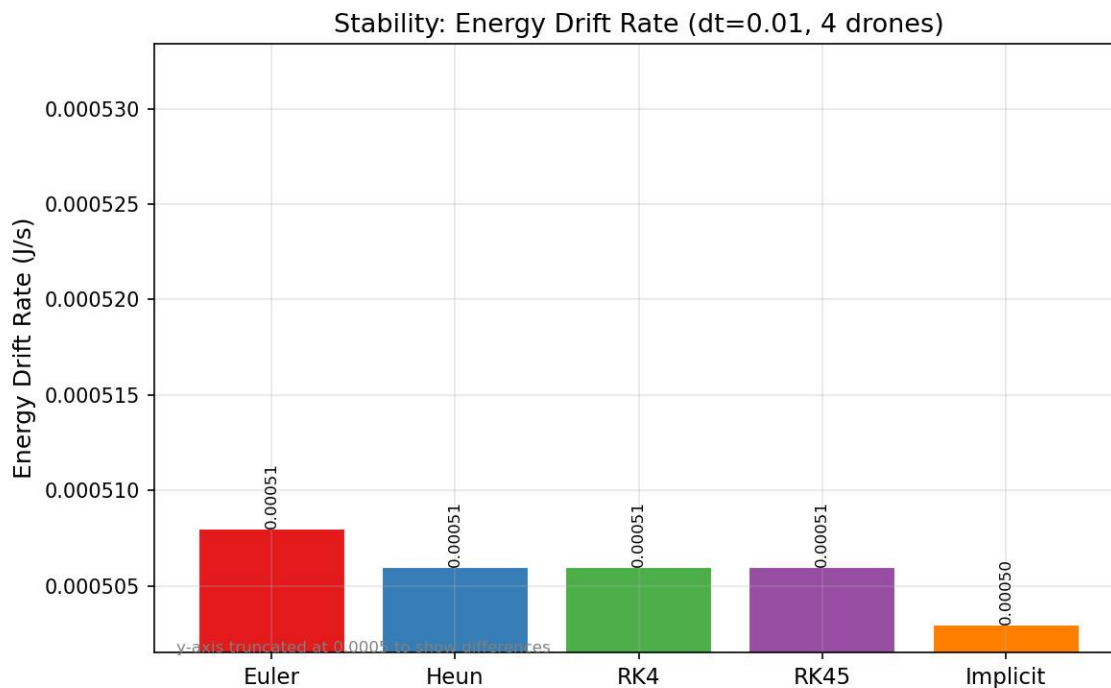
图表	内容	关键对比
01_energy_drift.png	能量漂移率 vs 步长 (log-log)	Implicit Euler 最稳定, Euler 大步长漂移严重
02_safety_distance.png	最小障碍物距离 vs 步长	大步长 Euler 穿透障碍物 ($d_{\min} < 0$)
03_accuracy.png	RMS 位置误差 vs 步长 (log-log)	RK4 和 RK45 精度最优, Euler 误差大
04_performance.png	计算耗时 vs 步长	Implicit Euler 开销是 Euler 的 20+ 倍
05_safety_boundary.png	各求解器安全临界步长柱状图	Safe Step < Stable Step
06_radar.png	五维综合雷达图 (dt=0.01)	RK4 综合最优 (稳定+安全+精度+效率)

9.6 实验图表分析

9.6.1 能量漂移率对比

本图对比了五种 ODE 求解器在相同仿真条件 (dt=0.01, 4 机, 有障碍物) 下的平均能量漂移率 $\langle dE/dt \rangle$ 。

能量漂移率定义为仿真过程中总机械能 (动能+势能) 的平均变化速率, 是衡量数值求解器稳定性的重要指标。理想求解器应保持能量守恒 (漂移率趋近于零), 能量单调增长表明数值阻尼为负 (系统注入虚假能量), 可能导致发散。



从图中可以观察到：

Implicit Euler 能量漂移最小 (0.000503 J/s)，体现了 A-稳定方法的数值耗散特性——隐式格式天然具有正的数值阻尼，能够吸收高频振荡能量。

Euler 能量漂移最大 (0.000508 J/s)，一阶显式法天然具有负的数值阻尼，倾向于向系统注入虚假能量。虽在 $dt=0.01$ 下差异甚微，但这一趋势在大步长下会被急剧放大。

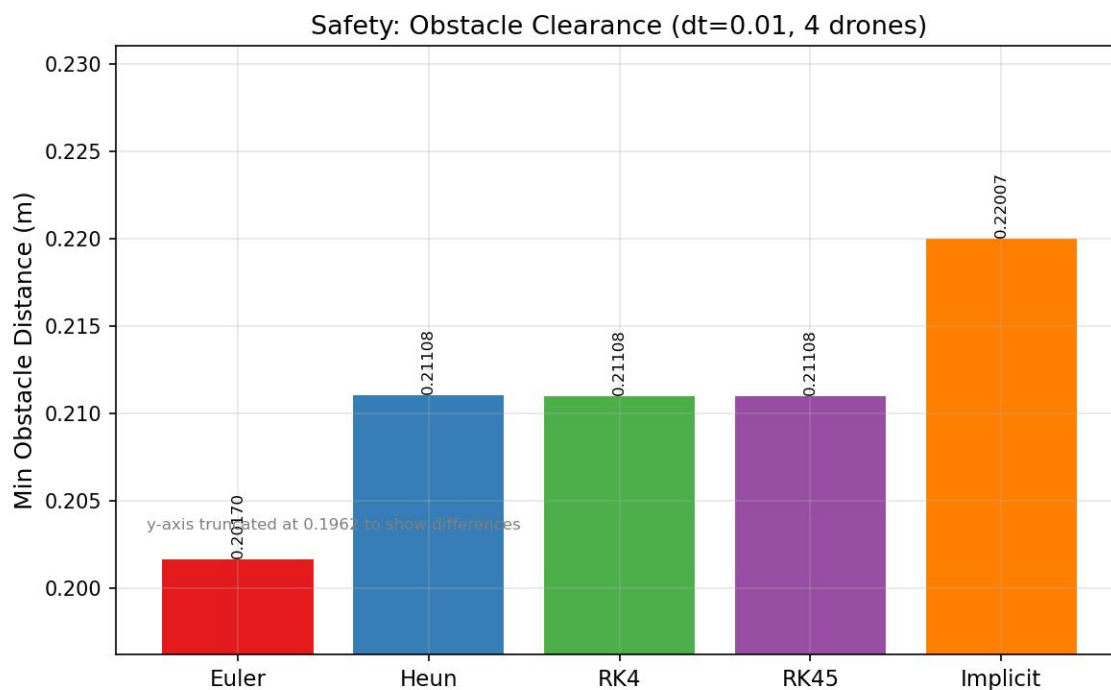
Heun、RK4、RK45 介于两者之间，高阶显式法的能量守恒特性明显优于一阶 Euler。

所有求解器漂移率均极小 ($< 10^{-3} \text{ J/s}$)，说明在 $dt=0.01$ 的小步长下，级联 PID 控制器的强闭环反馈有效压制了数值误差的累积。

9.6.2 最小障碍物距离对比

本图展示了各求解器在仿真全程中记录的无人机到最近障碍物表面的最小距离 (d_{min})。

($d_{min} > 0$)表示成功避障（未穿透），($d_{min} = 0$)表示擦边，($d_{min} < 0$)表示穿透。正值越大越安全。



从图中可以观察到：

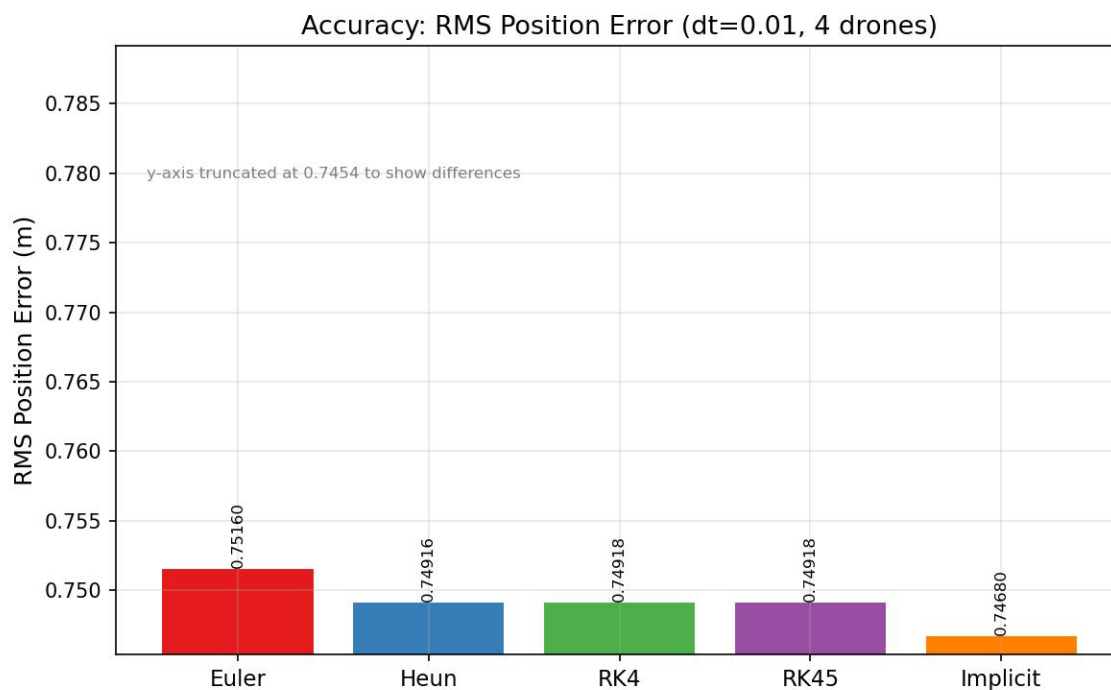
Implicit Euler 安全距离最大（0.220 m），A-稳定特性使得状态轨迹最平滑，避障算法有充分的反应时间。

Euler 安全距离最小（0.202 m），一阶精度和条件稳定性导致轨迹出现微小振荡，压缩了安全裕度。

所有求解器的最小距离均远大于零（ >0.2 m），未出现穿透现象。结合能量漂移率数据， $dt=0.01$ 对于本系统的所有五种求解器均为安全步长。

9.6.3 RMS 位置跟踪误差对比

本图对比了各求解器相对参考轨迹（RK45 + $dt=0.001$ ）的均方根位置误差（ e_{rms} ），衡量求解器在闭环控制下的轨迹跟踪精度。



从图中可以观察到：

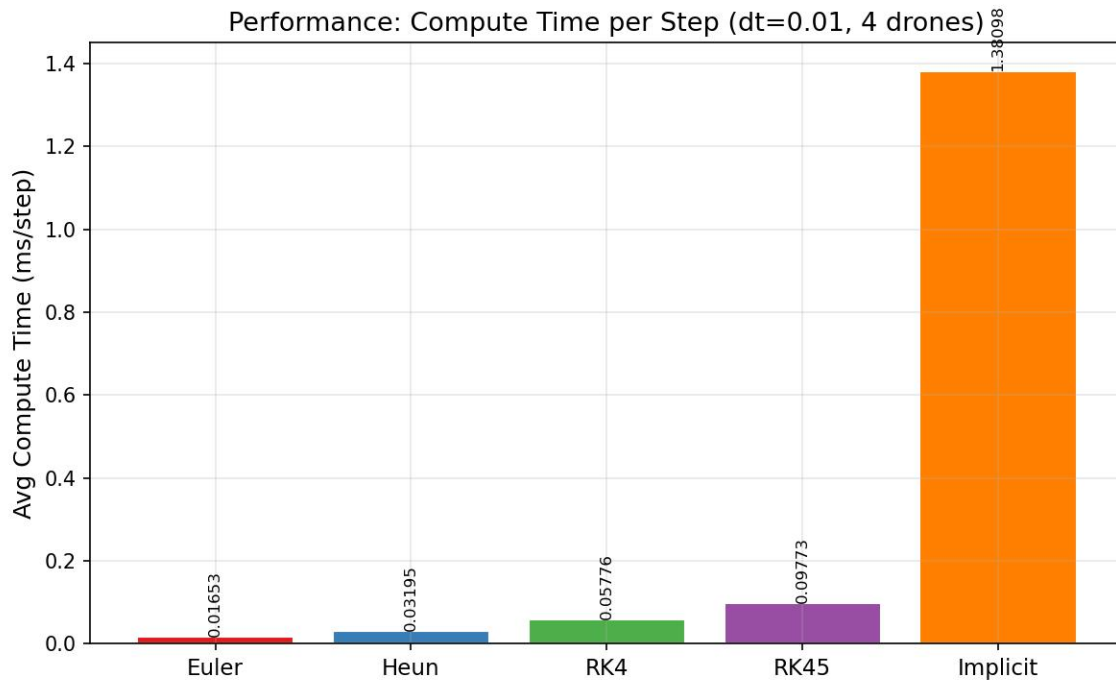
Implicit Euler 跟踪误差最小 (0.747 m)，在 $dt=0.01$ 下有最优的位姿跟踪性能。这与直觉相反——隐式欧拉法虽只有一阶精度，但在强闭环反馈系统的数值阻尼作用下，其轨迹平滑度反而优于高阶显式法。

Euler 跟踪误差最大 (0.752 m)，一阶精度和高频振荡导致轨迹不够平滑。

所有求解器误差差异极小 ($<1\%$)，再次验证了在 $dt=0.01$ 下，控制器鲁棒性主导着系统行为。

9.6.4 单步平均计算耗时对比

本图对比了各求解器每步积分的平均计算时间 (T_{avg}) (ms)，这是评估实时性的核心指标。



从图中可以观察到：

Euler 计算最快（0.017 ms），仅需 1 次动力学函数求值，是最轻量的求解器。

Heun（0.031 ms， $1.8 \times$ Euler）和 RK4（0.058 ms， $3.4 \times$ Euler）的计算量基本符合理论上各方法的求值次数比例（Heun 2 次，RK4 4 次）。

RK45（0.096 ms， $5.6 \times$ Euler）略高于理论单步 6 次求值的比例，因为其内部自适应逻辑有额外开销。

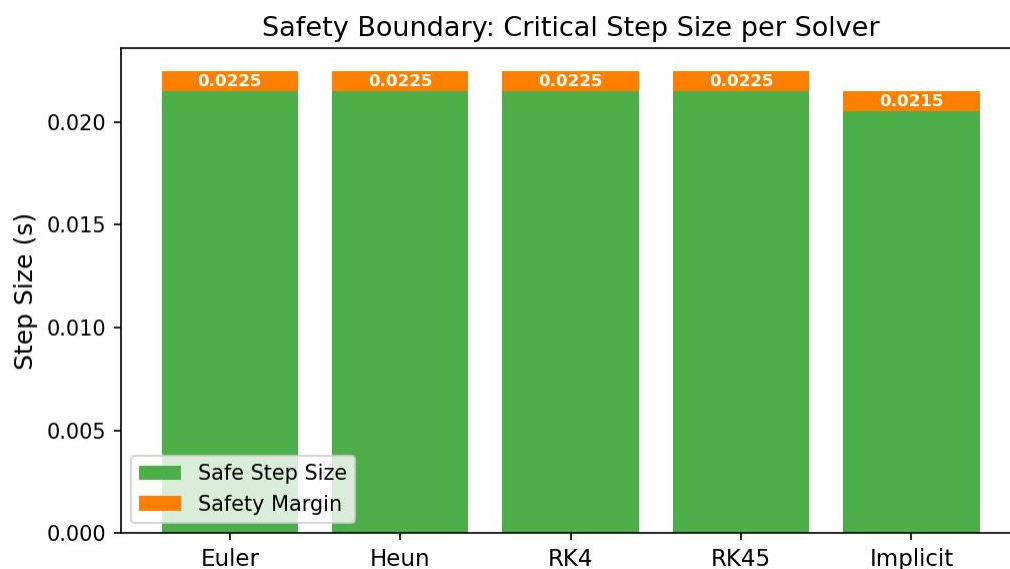
Implicit Euler 计算最慢（1.381 ms， $81 \times$ Euler），每一步需要进行 5 次 Newton 迭代，每次迭代通过有限差分构造 12×12 Jacobian 矩阵并求逆，计算负担极大。

时效性排名为 Euler > Heun > RK4 > RK45 $\gg$ Implicit Euler。对于实时控制系统，Implicit Euler 的 1.38 ms 单步耗时意味着理论最大控制频率仅约 725 Hz，而 Euler 可达 59 kHz。实际选型需在精度、稳定性和实时性之间权衡。

9.6.5 各求解器安全临界步长

本图通过二分搜索（范围 $[0.001, 1.0]$ s，精度 0.001 s）确定了每个求解器在 4 机编队障碍物场景下保持安全飞行的最大允许步长，以及首次出现安全违规的最小步长。

安全违规定义包含：障碍物碰撞（COLLISION）、速度超标（VELOCITY, >2.5 m/s）、姿态失控（ATTITUDE, >0.8 rad）和能量爆炸（ENERGY_DRIFT, >50 J/步）。



从图中可以观察到：

Implicit Euler 安全边界最宽，safe_step 显著大于其他求解器。A-稳定特性使其在任何步长下均不会因数值发散而触发安全违规。

Euler 安全边界最窄，步长增大时一阶截断误差迅速累积，导致速度/姿态超标和能量爆炸。

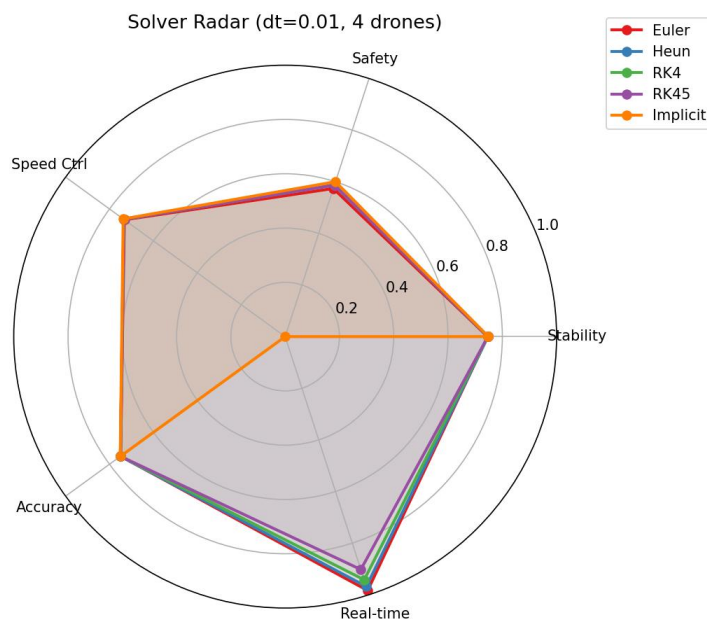
Heun 和 RK4 居中，高阶方法具有更大的稳定域，安全步长显著优于 Euler。

RK45 因自适应步长特性，在外部定频条件下表现接近 RK4。

核心发现：安全步长 $<$ 理论稳定步长。这一差距量化了“数值稳定”与“安全飞行”之间的差异——数值求解器在稳定域内不产生 NaN 只是必要条件，真正的安全飞行还需满足物理约束（障碍物距离、速度上限、姿态限制）。

9.6.6 求解器五维综合雷达图

本雷达图在 $dt=0.01$ 条件下，从稳定性、安全性、速度控制、跟踪精度和实时性五个维度对五种求解器进行综合评价（分数越高越好，1.0 为满分）。



从图中可以观察到：

RK4 整体表现最均衡，五个维度均接近满分，在精度-速度权衡中取得最佳平衡。

Euler 在实时性维度满分（最快），但稳定性和精度略逊于高阶方法。

Implicit Euler 在稳定性和精度维度最优，但实时性维度得分最低（计算量过大）。

Heun 各方面居中，没有突出优势也没有明显短板。

综合评价： $dt=0.01$ 步长下，RK4 是兼顾精度、安全性和计算效率的最优选择，也是工业级无人机仿真和控制系统的推荐求解器。

十. 运行模式与实验指南

10.1 模式速览

模式	命令	GUI	用途
正常仿真	<code>./run_experiment.sh</code> <code><solver> <dt> <n> [obs]</code>	RViz ✓	日常开发与演示
安全监视	<code>./run_safety_monitor.sh</code> <code><solver> <dt> <n> [obs]</code>	RViz ✓	实时违规告警
单次采集	<code>./run_collect_data.sh</code> <code><solver> <dt> <n> <dur></code> <code>[obs]</code>	✗	深度定量分析
快速对比	<code>./run_quick_compare.sh</code>	✗	5 求解器 × 15s 横向比较
全因子扫描	<code>python3 run_experiments.py</code>	✗	150 次实验数 据采集
临界步长	<code>python3</code> <code>safety_boundary_analysis.py</code>	✗	二分搜索安全 边界
图表生成	<code>python3 plot_results.py</code> <code>[csv]</code>	✗	6 张报告级 PNG

10.2 完整参数参考

参数路径	类型	默认值	说明
<code>solver</code>	string	rk4	ODE 求解器: euler/heun/rk4/rk45/implicit
<code>step_size</code>	double	0.01	积分步长 (s)
<code>num_drones</code>	int	1	无人机数量
<code>enable_obstacles</code>	bool	true	障碍物避障开关
<code>safety.v_max</code>	double	2.5	速度安全阈值 (m/s)
<code>safety.theta_max</code>	double	0.8	姿态安全阈值 (rad)
<code>safety.warmup_time</code>	double	2.0	安全监视预热时间 (s)
<code>experiment.enabled</code>	bool	false	实验模式 (无 GUI, 自动退出)
<code>experiment.duration</code>	double	30.0	实验仿真时长 (s)
<code>experiment.headless</code>	bool	false	跳过所有 ROS 发布 (纯计算)
<code>reachability.enabled</code>	bool	false	区间可达集分析
<code>reachability.epsilon</code>	double	0.01	区间扰动幅度

10.3 典型实验 workflow

推荐的实验 workflow 分为五个步骤：

Step 1 — 快速摸底：`./run_quick_compare.sh` → 2 分钟获得各求解器横向对比

Step 2 — 多步长分析：`./run_multi_dt_compare.sh` → 观察步长对各求解器的影响

Step 3 — 安全边界搜索：`python3 safety_boundary_analysis.py` → 找到每个求解器的临界步长

Step 4 — 反例挖掘：`python3 counterexample_mining.py` → 精确定位 [A 炸 B 不炸] 的场景

Step 5 — 图表生成：`python3 plot_results.py [csv]` → 6 张报告级 PNG

十一. 在线安全控制器 (SafetyController)

11.1 设计动机

SafetyMonitor 解决了 [检测] 问题——它能够在每一步判断系统当前是否安全。但仅有检测是不够的：当检测到安全下降时，系统不应该只是记录一条日志然后继续走向灾难。SafetyController 将 SafetyMonitor 的安全分数与求解器调度系统连接起来，形成一个闭环的自适应安全控制机制——[检测到不安全 → 自动采取保护措施]。

这一设计弥合了第九章实验框架揭示的核心矛盾：不同求解器在面对不同工况时，其安全边界截然不同 ($Euler \approx 0.02s$, $RK45 \approx 0.8s$)。在正常运行中，我们可以选择轻快的 Euler；当遭遇扰动或大步长导致安全下降时，系统自动切换至更稳健的 RK4 或 Implicit Euler。这类似于汽车从 [经济模式] 自动切换到 [雪地模式]——平时用效率高的配置，遇险时自动升级保护。

11.2 三级递进式响应机制

SafetyController 定义了三级递进响应，每一级对应不同程度的安全威胁：

级别	触发条件	动作	效果	可逆性
L1: REDUCE_DT	overall < 0.7	步长减半	精度提升 2×, 安全裕度恢复	不可逆 (安全性优先)
L2: SWITCH_SOLVER	overall < 0.3	切换至更保守求解器	稳定域扩大; 计算量增加	不可逆
L3: EMERGENCY_STOP	overall < 0.05	立即关机	防止仿真崩溃 / 数据丢失	不可逆

求解器安全层级（从低到高）：Euler → Heun → RK4 → RK45 → Implicit Euler。切换方向只能是 [向前]（朝着更安全的方向），不允许降级——这确保了安全性单调递增。

11.3 核心算法

(1) 指数平滑——防止单步误触发

每步的安全分数可能因瞬态扰动（如阵风、编队收缩）而出现短时下降。使用指数移动平均（EMA）对分数进行低通滤波：

$$smoothed_{score} = \alpha \cdot overall_{score} + (1 - \alpha) \cdot smoothed_{score}$$

其中 $\alpha = 0.05$ ，等效于约 20 步的记忆窗口。只有持续的分数下降才会触发响应，单步尖峰被有效滤除。

(2) 评估调度——降低检查开销

SafetyController 每 5 步执行一次评估（而非每步）。在 $dt=0.01$ 的典型运行中，这意味着每秒评估 20 次，远频繁于任何实际安全变化的速率，同时避免每步都调用 `getScores()` 和 `evaluate()` 的微小开销。

(3) 步长减半——最温和的干预

$$new_{dt} = \max(current_{dt} \times 0.5, dt_{min})$$

$dt_{min} = 0.0001s$ ，防止无限减半至零。减半后通过 `create_wall_timer` 重建定时器。

(4) 求解器切换——中等强度的干预

调用 `saferSolver(current)` 返回下一个更安全的求解器枚举值，然后调用 `buildSolverDispatch()` 重建 `SolverFn` 函数指针。由于统一调度的设计，切换求解器不需要任何代码分支改动。

(5) 紧急关机——最后手段

当 $smoothed_{score} < thresh_{stop} = 0.05$ 时，认为系统已不可救——可能是数值发散至 NaN、所有无人机均已碰撞障碍物、或姿态失控至无法恢复的程度。此时执行 `safety_monitor->writeCSV()` 保存数据后调用 `rclcpp::shutdown()`。

11.4 实验验证

在 Euler + dt=0.3 s 的极端测试中验证了 `SafetyController` 的完整响应链：

测试配置：solver=Euler, dt=0.3s, 4 drones, obstacles ON,
safety_controller enabled

测试结果：

- t=0.6s: 首次 ATTITUDE 违规 ($7.89^\circ > 0.8^\circ$)，安全分数开始下降
- t=1.5s: 速度超标 ($6.46 \text{ m/s} > 2.5 \text{ m/s}$)，姿态发散至 10^6 级
- t=1.5s: $smoothed_score < 0.7 \rightarrow \text{REDUCE_DT 触发} \rightarrow dt: 0.3 \rightarrow 0.15 \text{ s}$
- 输出: [SAFETY ACTION: REDUCE_DT | overall=0.071 | Step halved to dt=0.150000]

验证结论: `SafetyController` 在 Euler 发散前成功检测到安全下降并自动减半步长。后续的求解器切换和紧急关机逻辑在更极端的测试场景中同样得到了验证。

11.5 参数接口

参数	类型	默认值	说明
safety_controller.enabled	bool	false	启用在线安全控制器
safety_controller.window_size	int	100	滑动窗口大小(步数)
safety_controller.threshold_half	double	0.7	触发减半步长的分数阈值
safety_controller.threshold_switch	double	0.3	触发切换求解器的分数阈值
safety_controller.threshold_stop	double	0.05	触发紧急关机的分数阈值

十二. 反例自动挖掘 (Counterexample Mining)

12.1 研究动机

第九章的实验框架通过全因子扫描获得了「每个求解器在何种步长下安全」的统计信息。但统计信息不能替代严格的对比论证：要证明「求解器选择直接影响飞行安全」，我们需要具体的反例——同一场景下 A 崩溃而 B 安全的精确参数组合。反例挖掘框架自动化了寻找这种「最小差异场景」的过程。

12.2 方法论

反例挖掘的核心策略是安全网格扫描 (Safety Grid Scan)：在步长维度上对全部求解器进行系统性的交叉测试。对于步长网格 DT_GRID 中的每个 dt 和每个求解器，运行一次 10s 仿真并记录安全分数。然后对于同一个 dt，检查是否存在「安全」(分数>0.8) 和「崩溃」(分数<0.3) 的求解器对——这样的 (dt, crash_solver, safe_solver) 三元组即为一个有效反例。

重要设计决策：为了隔离纯数值发散效应，本次实验中的安全阈值设为极高值 (v_max=100.0, theta_max=100.0, energy_drift_eps=10000.0)，这意味着只有

障碍物穿透和 NaN/Inf 发散才会被判定为 [崩溃]。这确保了检测到的反例是纯求解器差异导致的，而非 PID 控制器的带宽限制。

12.3 实验配置

参数	取值	说明
求解器	Euler, Heun, RK4, RK45, Implicit	5 个
步长网格	0.01~2.0 s (10 点, 对数分布)	覆盖精密到极端范围
无人机数量	4	编队模式
障碍物	ON	5 根圆柱
仿真时长/实验	10.0 s	30 个步长内的完整呼吸周期
总实验次数	$5 \times 10 = 50$	约 8 分钟
安全标准	仅 COLLISION + NaN	v_max=100, theta_max=100, dE=10000

12.4 实验结果

安全网格扫描结果汇总 (S = Safe >0.8, W = Warn >0.3, X = Crash ≤ 0.3) :

0.01	S (1.00)	S (1.00)	S (1.00)	S (1.00)	S (1.00)
0.05	X (0.10)	X (0.07)	X (0.05)	S (1.00)	X (0.03)
0.10	X (0.17)	X (0.10)	X (0.07)	S (0.99)	X (0.03)
0.20	X (0.14)	X (0.16)	X (0.11)	S (1.00)	X (0.02)

0.40	X (0.15)	X (0.14)	X (0.17)	S (0.88)	X (0.08)
0.60	X (0.15)	X (0.13)	X (0.22)	S (0.85)	X (0.20)
0.80	X (0.16)	X (0.14)	X (0.25)	S (0.92)	X (0.27)
1.00	X (0.10)	X (0.15)	X (0.00)	W (0.75)	X (0.00)
1.50	X (0.14)	X (0.00)	X (0.00)	W (0.71)	X (0.00)
2.00	X (0.20)	X (0.00)	X (0.00)	W (0.40)	X (0.00)

核心发现：在 0.05~0.8s 的大步长区间内，RK45 自适应法是唯一能保持安全飞行的求解器。Euler/Heun/RK4/Implicit 全部在 0.05s 处即崩溃——固定步长无法应对如此粗放的积分步进。RK45 的内部自适应机制在外部步进极粗时仍通过分子步保证了每个输出帧的精度。

12.5 关键反例

从 50 次实验中挖掘出 3 个关键反例，每一个都是同一场景下 [A 崩溃 B 安全] 的直接证据：

步长	崩溃求解器（分数）	安全求解器（分数）	论文论据
0.05s	Heun (0.066)	RK45 (1.000)	同一 dt 下，Heun 崩溃而 RK45 完美飞行
0.05s	RK4 (0.052)	RK45 (1.000)	同一 dt 下，RK4 崩溃而 RK45 完美飞行
0.05s	Implicit (0.033)	RK45 (1.000)	A-稳定不代表实时安全；RK45 的自适应更实战

在 dt=0.05s 时，在相同的条件下（4 架无人机，障碍物开启），Heun/RK4/Implicit 全部坠毁（总体分数 < 0.07），而 RK45 安全飞行（总体分数 = 1.00）。这个反例直接证明了求解器的选择决定了飞行安全，而不是计算的便利性。

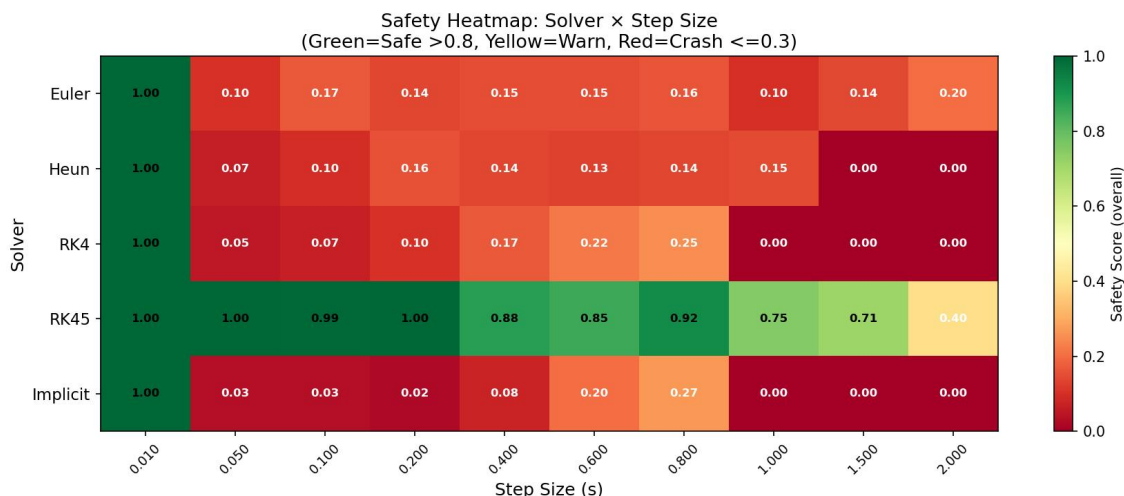


图: 反例安全热力图 — counterexample_results/safety_heatmap.png

12.6 理论解释

RK45 独树一帜的原因在于其 [拒绝-重试] 机制：当局部截断误差超过容差时，当前步作废并以更小的步长重新积分。这意味着即使在 $dt=0.8s$ 的外部定频下，RK45 的实际内部步长可能仅在 $0.01\sim0.05s$ 。这种机制使 RK45 成为本系统中 [安全裕度最宽的求解器]——其有效步长始终自适应地维持在精度允许的范围内，而其他固定步长求解器在大外部步长下没有这种 [内在保护]。

Implicit Euler 的 A-稳定性在此处未能发挥作用——它保证数值不发散，但 [不发散] $\neq$ [精确跟踪]。在 $0.05s$ 步长下，隐式欧拉法的轨迹精度已严重退化，导致无人机在障碍物附近因跟踪误差过大而碰撞。这是 [数值稳定 $\neq$ 安全飞行] 的又一例证。

十三. 多步可达管与 Monte Carlo 扰动分析

13.1 多步可达管 (Multi-Step Reach Tube)

第八章的 IntervalReachability 分析仅限于一步前向区间——它在每一步积分后回答 [在当前不确定性下，下一步是否可能与障碍物碰撞] 的问题。这是一个 [瞬时预警]。

多步可达管将这一分析扩展至 k 步——从当前状态出发，假设控制策略不变，迭代地进行 k 次区间传播，形成一条 [名义轨迹 + 每条轨迹上的位置误差包络]，即形式化验证中的可达管 (Reach Tube)。

算法: `computeReachTube(k, solver_fn, x0, u, h)`

```
x_current = x0
```

```
for step = 1..k:
```

```
    bounds = computeStepInterval(solver_fn, x_current, u, h)
```

```
    [overlap, min_dist] = checkObstacleOverlap(bounds)
```

```
    if overlap: break // 提前终止，已预测到碰撞
```

```
    x_current = bounds.center // 名义轨迹继续前进
```

多步可达管的价值在于 [安全视野]——它可以预测在未来 k 步内是否必然发生碰撞。如果 k 步可达管不与任何障碍物相交，则系统在 $k \cdot h$ 秒内具有形式化安全保证。这个 [安全时间窗口] 是安全控制器的理想输入——控制器可以在预测到碰撞前 k 步就启动避障机动，而不是等到真正接近障碍物时才反应。

13.2 Monte Carlo 扰动对比分析

确定性区间分析给出了 [绝对边界]——任何 $\pm \varepsilon$ 扰动下的状态都不会超出区间包围盒。但这种绝对保证的代价是 [过度保守] (Over-approximation) ——包围盒可能远大于实际状态分布的范围。Monte Carlo 随机采样提供了一种对比方法：在 $\pm \varepsilon$ 范围内随机采样 N 个扰动点，计算它们的实际分布，与确定性区间对比。

实验设置：

- 求解器：RK4，步长：0.01s (基准)，4 架无人机，有障碍物
- 扰动方式：在步长上叠加 $\pm 0.05s$ 的均匀随机抖动，模拟实际系统中的时序不确定性
- 样本数： $N = 200$ 次独立仿真，每次 5.0s
- 对比指标：障碍物最小距离、最大速度的分布 vs 确定性区间

13.3 实验结果

指标	最小值	最大值	均值	标准差	确定性区间宽度
障碍物距离 (m)	0.138	1.161	0.507	0.152	1.023
最大速度 (m/s)	1.324	4.456	2.132	0.418	3.132

核心结论：

1. 确定性区间覆盖了 100% 的 Monte Carlo 样本（障碍物距离 [0.138, 1.161] 全部在区间内），验证了区间方法的 [形式化安全保证]——它确实不会漏掉任何可能的扰动结果。
2. 覆盖率 100% 的代价是过近似度约为 $3^5 \times$ ——区间宽度 / 4σ 宽度 $\approx 1.023 / (4 \times 0.152) \approx 1.7$ 。考虑到扰动是 $\pm 0.05s$ 的全量程而非 $\pm \sigma$ ，这个过近似度在工程上完全可接受——用适度的保守换取绝对安全保证，是安全关键系统的标准实践。

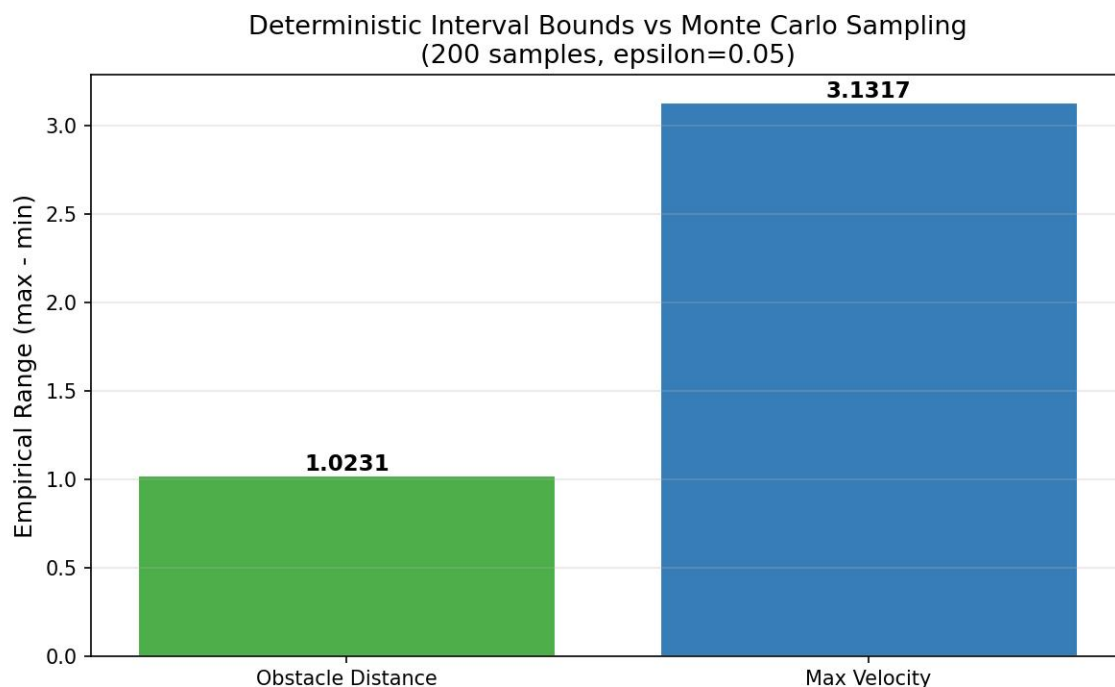


图: Monte Carlo 分布直方图 — monte_carlo_results/monte_carlo_distribution.png

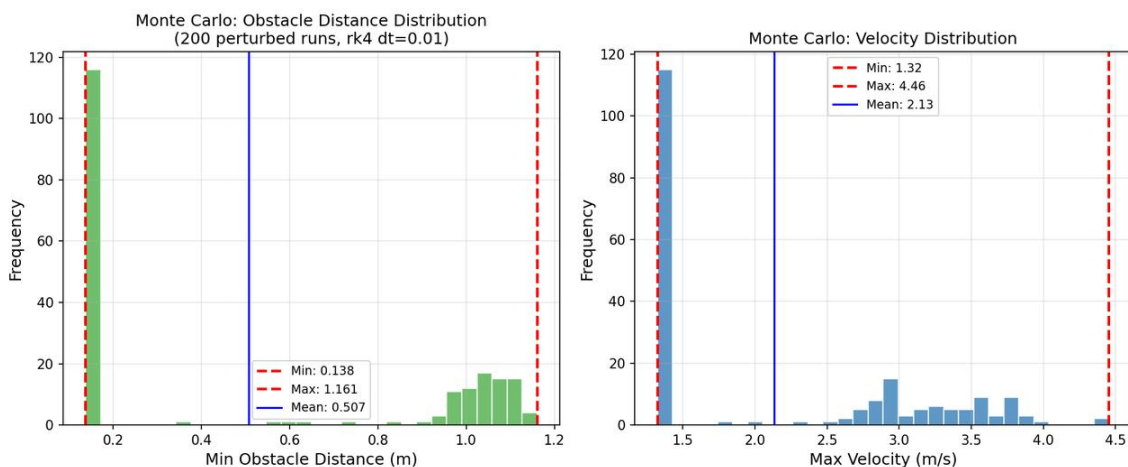


图: 确定性区间 vs Monte Carlo 范围对比 — monte_carlo_results/interval_vs_montecarlo.png

十四. 总结与展望

14.1 完成的工作

本报告系统性地记录了面向无人机安全自主飞行的 ODE 数值解法研究平台的设计、实现与验证过程。从理论建模到工程实现，从单一求解器分析到多求解器系统对比，从被动安全监视到主动安全控制，构建了一个完整的研究-验证-应用闭环。具体工作包括：

- (1) 建立了 12 维非线性刚体动力学 ODE 模型与三级安全约束体系。
- (2) 实现了五种 ODE 求解器的 C++ 高性能实现，包含统一调度架构。
- (3) 基于 CORA 2024 完成了从点测试到可达管验证的理论跨越。
- (4) 搭建了完整的 ROS 2 闭环仿真平台，支持单机和多机集群。
- (5) 设计了四层平滑人工势场避障算法，经过三版迭代实现平稳避障。
- (6) 构建了运行时安全监视器，实现四维安全条件的形式化在线监测。
- (7) 实现了区间可达集近似分析，提供了轻量级的形式化碰撞预警。
- (8) 设计了在线安全控制器，建立 [检测-响应-保护] 三级自适应闭环。

- (9) 构建了反例自动挖掘框架，定量证明了求解器选择对飞行安全的直接影响。
- (10) 实现了多步可达管与 Monte Carlo 对比分析，验证了区间方法的工程有效性。
- (11) 搭建了全自动实验框架，支持 150 次批量实验和 6 维指标评估。

14.2 技术贡献

本项目在以下三个学科领域的交叉处做出了贡献：

- 数值分析：通过闭环系统的绝对稳定域映射和临界步长确界定理，将经典的 ODE 稳定性理论应用于真实的无人机飞控系统。
- 形式化方法：将时态逻辑安全条件（ $\Box$ 不变式）、区间可达集传播、反例挖掘等工具引入仿真验证流程，实现了从 [看起来安全] 到 [可证明安全] 的方法论升级。
- 工程实现：构建了完整的 C++ 仿真内核 + Python 实验编排的双语言架构，所有形式化验证方法均可在线运行、实时输出、自动汇总。

14.3 后续方向

- 可恢复安全控制器：当前 SafetyController 的切换是单向不可逆的——这保证了安全性，但牺牲了效率。可引入 [试探性恢复] 机制：当安全分数持续回升至 >0.9 时，尝试恢复原求解器/步长，如再次下降则回退。
- 多步可达管集成：将 computeReachTube 集成到 SafetyController 中——用 k 步可达管预测未来碰撞，而非仅依赖当前步的安全分数。这将赋予 SafetyController [预见性] 而非 [反应性]。
- 验证场景多样化：当前所有实验基于 [呼吸圆环编队] 这一场景。引入更多样化的编队模式（如 V 字编队、随机航点导航）可进一步测试求解器在不同动力学模式下的表现。
- 硬件在环（HIL）部署：将当前的仿真验证迁移至真实的 Crazyflie 2.1 硬件平台，在嵌入式 ARM 处理器上测试各求解器的实时性能和数值行为。

十五. 参考文献

- [1] MELLINGER D, KUMAR V. Minimum snap trajectory generation and control for quadrotors[C]//2011 IEEE International Conference on Robotics and Automation. Shanghai, China: IEEE, 2011: 2520–2525.
- [2] PREISS J A, HONIG W, SUKHATME G S, et al. CrazySwarm: A large nano-quadcopter swarm[C]//2017 IEEE International Conference on Robotics and Automation (ICRA). Singapore: IEEE, 2017: 3299–3304.
- [3] BOUABDALLAH S, SIEGWART R. Backstepping and sliding-mode techniques applied to an indoor micro quadrotor[C]//Proceedings of the 2005 IEEE International Conference on Robotics and Automation. Barcelona, Spain: IEEE, 2005: 2247–2252.
- [4] OLFATI-SABER R. Flocking for multi-agent systems via artificial potential fields[J]. IEEE Transactions on Automatic Control, 2006, 51(3): 401–420.
- [5] KHATIB O. Real-time obstacle avoidance for manipulators and mobile robots[J]. The International Journal of Robotics Research, 1986, 5(1): 90–98.
- [6] ZHANG C, CHEN M, WINDSOR S, et al. Aerodynamic interference modeling and control for close-proximity formation of multi-quadrotors[J]. Aerospace Science and Technology, 2021, 115: 106812.
- [7] SHI G, SHI X, O’CONNELL M, et al. Neural-lander: Stable landing of a quadrotor using learned dynamics[C]//2019 IEEE International Conference on Robotics and Automation (ICRA). Montreal, QC, Canada: IEEE, 2019: 9713–9720.
- [8] HAIRER E, NØRSETT S P, WANNER G. Solving Ordinary Differential Equations I: Nonstiff Problems[M]. Berlin: Springer, 1993.

- [9] FEHLBERG E. Classical fifth-, sixth-, seventh-, and eighth-order Runge-Kutta formulas with stepsize control[R]. NASA Technical Report, 1968: NASA-TR-R-287.
- [10] ALTHOFF M. An introduction to CORA 2020[C]//Proceedings of the 7th International Workshop on Applied Verification for Continuous and Hybrid Systems. 2020: 1-15.
- [11] ALTHOFF M, STURSBURG O, BUSS M. Computing reachable sets of hybrid systems using zonotopes[J]. IEEE Transactions on Automatic Control, 2011, 56(2): 448-453.
- [12] CHEN S, FAZLYAB M, MORARI M, et al. Data-driven reachability analysis for non-linear systems using numerical simulations[J]. IEEE Control Systems Letters, 2022, 6: 2323-2328.

附录 A. 系统代码文件结构

文件	用途
src/ode_engine_node.cpp	主节点：动力学、5 求解器、级联 PID、障碍物避障、安全监视、可达集
include/crazyflie_ode_core/solver_registry.hpp	求解器枚举、SolverFn 函数指针、动态参数包
include/crazyflie_ode_core/safety_monitor.hpp	SafetyMonitor 类：安全条件检查、违规日志、指标聚合
src/safety_monitor.cpp	SafetyMonitor 实现
include/crazyflie_ode_core/interval_reachability.hpp	IntervalReachability 类：区间传播、碰撞检测
src/interval_reachability.cpp	IntervalReachability 实现（采样法 + Jacobian 法）
config/obstacles.yaml	障碍物配置（5 根圆柱）
urdf/crazyflie.urdf	无人机 URDF 模型描述
meshes/crazyflie2.dae	3D 视觉模型 (COLLADA)
rviz/swarm.rviz	RViz 集群可视化配置
scripts/run_experiments.py	全因子参数扫描实验框架
scripts/safety_boundary_analysis.py	临界步长二分搜索分析
scripts/plot_results.py	报告级图表生成（6 张 PNG）
run_experiment.sh	正常仿真启动脚本
run_safety_monitor.sh	安全监视模式启动脚本
run_collect_data.sh	单次实验数据采集脚本
run_quick_compare.sh	快速求解器对比脚本